

Universitatea Națională de Știință și Tehnologie POLITEHNICA București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

*Platformă Integrată pentru Gestionarea Resurselor și Activităților
Universitare*

Proiect de diplomă

prezentat ca cerință parțială pentru obținerea titlului de
Inginer în domeniul Electronică, Telecomunicații și Tehnologia Informației
programul de studii de licență Microelectronică, optoelectronică și
nanotehnologii

Conducător(i) științific(i)
Conf. dr. ing. Petru-Lucian MILEA

Absolvent
Robert TUDOSE

Anul 2026

TEMA PROIECTULUI DE DIPLOMĂ

a studentului **TUDOSE C. Robert, 443E**

1. Titlul temei: Platformă Integrată pentru Gestionarea Resurselor și Activităților Universitare

2. Descrierea temei:

Se va realiza o aplicație web pentru eficientizarea fluxului operațional zilnic din cadrul facultății, cu resursele și activitățile aferente: utilizatori, săli, cursuri, orare și activități de secretariat. Sistemul va fi construit pe o arhitectură modernă de tip SPA (Single Page Application), oferind o experiență fluidă și securizată. Accesul va fi diferențiat pe baza a patru roluri distincte (Administrator/Admin, Profesor, Student, Reprezentant Extern), fiecare beneficiind de o interfață personalizată, adaptat nevoilor specifice. Platforma va asista generarea orarului, prin detectarea automată a suprapunerilor pentru săli, cadre didactice, grupe de studenți și discipline. Administratorii vor putea alocă studenții pe ani, specializări și grupe, vor putea defini atribute specifice pentru săli (capacitate, echipamente, disponibilitate, rezervare de către terți). Activitățile de secretariat vor permite programări și solicitări online. Interfețele utilizatorilor vor fi adaptabile automat pentru dispozitivul utilizat și lumina ambiantă. Implementarea se va face folosind utilizând stiva tehnologică MERN (MongoDB: Bază de date nerelațională, Express: Server web NodeJS, React: Bibliotecă frontend JavaScript, Node: Server web JavaScript).

3. Contribuția originală:

Dezvoltarea completă a platformei web demonstrative, cu funcționalitățile descrise, utilizând stiva MERN.

4. Resurse și bibliografie:

Programarea Calculatoarelor, Structuri de Date și Algoritmi, Programare Obiect-Orientată, Baze de Date, Proiectarea Interfețelor pentru Baze de Date, Tehnologii de Programare în Internet.

5. Data înregistrării temei: 2026-03-23 22:38:39

Conducător(i) lucrare,

Conf. dr. ing. Petru-Lucian MILEA

Student,

TUDOSE C. Robert

Director departament,

Decan,

Prof. dr. ing. Mihnea UDREA

Cod Validare: **72e414cc8f**

Declarație de onestitate academică

Prin prezenta declar că lucrarea cu titlul “*Platformă Integrată pentru Gestionarea Resurselor și Activităților Universitare*”, prezentată în cadrul Facultății de Electronică, Telecomunicații și Tehnologia Informației a Universității Naționale de Știință și Tehnologie POLITEHNICA București ca cerință parțială pentru obținerea titlului de *Inginer* în domeniul *Electronică, Telecomunicații și Tehnologia Informației*, programul de studii *licență Microelectronică, Optoelectronică și Nanotehnologii* este scrisă de mine și nu a mai fost prezentată niciodată la o facultate sau instituție de învățământ superior din țară sau străinătate.

Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referințe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și fac referință la sursă. Reformularea în cuvinte proprii a textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare.

Declar că toate rezultatele simulărilor, experimentelor și măsurărilor pe care le prezint ca fiind făcute de mine, precum și metodele prin care au fost obținute, sunt reale și provin din respectivele simulări, experimente și măsurători. Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

București, 19.06.2026

Absolvent *Robert TUDOSE*



(semnătura în original)

Cuprins

Introducere	17
Capitolul 1. Tehnologii și instrumente software	19
Capitolul 1.1. Arhitectura stivei MERN	19
Capitolul 1.1.1. Ecosistem lingvistic unificat, sau stive tradiționale.....	19
Capitolul 1.2. Tehnologii Frontend	19
Capitolul 1.2.1. DOM-ul virtual și reutilizarea de componente.....	20
Capitolul 1.2.2. Arhitectura rețelei și interceptoare HTTP.....	20
Capitolul 1.2.3. Motoare de stilizare CSS-in-JS și standarde de Accesibilitate (WCAG).....	21
Capitolul 1.3. Tehnologii Backend	21
Capitolul 1.3.1. I/O asincron și arhitectura buclei de evenimente.....	21
Capitolul 1.3.2. Structura criptografică și autentificarea fără stare.....	22
Capitolul 1.4. Bază de date și modelarea datelor	23
Capitolul 1.4.1. Necesitatea unei scheme flexibile	23
Capitolul 1.5. Instrumente de dezvoltare si implementare	23
Capitolul 2. Arhitectura sistemului si modelarea datelor	25
Capitolul 2.1. Arhitectură client-server decuplată	25
Capitolul 2.1.1. Protocolul de comunicare RESTful API.....	25
Capitolul 2.2. Controlul accesului bazat pe roluri (RBAC – Role-Based Access Control)	25
Capitolul 2.2.1. Securitatea rutelor cu ajutorul Middleware cu JWT.....	25
Capitolul 2.3. Strategii relaționale într-un mediu non-relațional	26
Capitolul 2.3.1. Integritate referențială sau încorporarea datelor.....	26
Capitolul 2.3.2. Joncțiuni cu ajutorul metodei populate() a Mongoose.....	27
Capitolul 2.3.3. Agregări relaționale complexe folosind operatorul \$lookup.....	27
Capitolul 2.4. Securitatea infrastructurii și containerizarea (DevOps).....	28
Capitolul 2.4.1. Orchestrarea microserviciilor folosind Docker.....	28
Capitolul 2.4.2. Securitatea containerului și principiul privilegiilor minime.....	29
Capitolul 2.5. Gestionarea stărilor globale și structura componentelor.....	30
Capitolul 2.5.1. Limitarea transmiterii datelor prin componente intermediare.....	30
Capitolul 2.5.2. Implementarea API-ului Context.....	30
Capitolul 2.6. Securitate la nivel de aplicație și pipeline-uri de middleware.....	30
Capitolul 2.6.1. Arhitectura CORS.....	30
Capitolul 2.6.2. Ciclul de viață al solicitărilor middleware Express.....	31
Capitolul 2.7. Diagrama Entitate-Relație (ERD) și structura datelor.....	31
Capitolul 2.8. Fluxul arhitectural și topologia Sistemului.....	32
Capitolul 3. Implementarea Algoritmilor de bază	35
Capitolul 3.1. Algoritm de detecție a suprapunerilor pe 4 niveluri	35
Capitolul 3.1.1. Refactorizare arhitecturală și principiul DRY.....	36
Capitolul 3.2. Inițializare automatizată a mediului și anonimizarea datelor.....	37

Capitolul 3.2.1. Paradigma inițializării.....	37
Capitolul 3.2.2. Anonimizarea dinamică a datelor și respectarea confidențialității.....	38
Capitolul 3.2.3. Logică de afaceri condiționată și integritate referențială.....	39
Capitolul 3.2.4. Mecanisme asincrone de ștergere a datelor.....	39
Capitolul 3.3. Modulul de rezervări a sălilor și optimizarea interogărilor temporale	40
Capitolul 3.4. Generarea dinamică de intervale orare pentru rezervări la secretariat	41
Capitolul 3.4.1. Arhitectură bazată pe configurare și generator dinamic de intervale orare.....	41
Capitolul 3.4.2. Filtrarea proactivă a suprapunerilor	43
Capitolul 3.5. Automatizare în fundal și întreținerea bazei de date	44
Capitolul 3.5.1. Necesitatea unui sistem autonom de mentenanță	44
Capitolul 3.5.2. Schimbările de stare bazate pe timp și execuția în fiecare oră	44
Capitolul 3.5.3. Pipeline de agregare pentru evaluare temporală	44
Capitolul 3.6. Hashing criptografic și autentificare fără stări.....	45
Capitolul 3.6.1. Apărare împotriva vulnerabilităților criptografice.....	45
Capitolul 3.6.2. Factorii de lucru algoritmici și fenomenul de salting.....	47
Capitolul 3.7. Validare temporală între colecții.....	47
Capitolul 3.7.1. Provocarea adusă de planificare cu contexte multiple.....	47
Capitolul 3.7.2. Realizarea algoritmică a matricei.....	47
Capitolul 4. Interfața grafică și experiența utilizatorului	49
Capitolul 4.1. Arhitectura componentelor și administrarea stărilor	49
Capitolul 4.2. Material UI și design reactiv	50
Capitolul 4.3. Optimizarea traficului de date și paginare gestionată de server	52
Capitolul 4.3.1. Problemele de randare a DOM (Document Object Model)	52
Capitolul 4.3.2. Pipeline-uri de agregare ale bazei de date	53
Capitolul 4.3.3. Integrarea în client și re-randarea dinamică a componentelor	54
Capitolul 4.4. Gestionarea dinamică a erorilor și interceptarea <code>18n</code>	55
Capitolul 4.4.1. Separarea logicii de bază a platformei și a limbii de prezentare	55
Capitolul 4.4.2. Interceptarea pe parte de client și maparea dinamică a cheilor	56
Capitolul 4.4.3. Injecție dinamică a parametrilor folosind <code>react-18next</code>	56
Capitolul 4.5. Vizualizarea datelor și interfața administrativă.....	58
Capitolul 4.5.1. Scopul analizei vizuale în gestiunea resurselor.....	58
Capitolul 4.5.2. Agregarea datelor în backend (<code>\$group</code>).....	59
Capitolul 4.5.3. Randare pe partea de client folosind diagrame Material UI.....	59
Capitolul 4.6. Micro-interacțiuni și îmbunătățiri de accesibilitate.....	61
Capitolul 4.6.1. Protecții impuse de stare pentru acțiuni distructive.....	61
Capitolul 4.6.2. Sincronizarea ferestrei de vizualizare a utilizatorului și bucle de feedback.....	61
Capitolul 4.7. Securitatea pe partea de client și arhitectura de securizare a rutelor.....	62
Capitolul 4.7.1. Necesitatea controlului accesului într-o aplicație full-stack.....	62
Capitolul 4.7.2. Componenta <code><ProtectedRoute></code>	62
Capitolul 4.7.3. Implementare în Router-ul React.....	63
Capitolul 4.8. Validare a datelor pe partea de client și prevenirea erorilor.....	64

Capitolul 4.8.1. Principiul apărării în profunzime.....	64
Capitolul 4.8.2. Constrângeri proactive ale interfeței utilizatorului.....	64
Capitolul 4.8.3. Validare logică pe bază de stări.....	65
Concluzii	67
Bibliografie	69

Lista figurilor

Figura 2.1 – Execuția stadiului de agregare \$lookup pentru asamblarea datelor relaționale	28
Figura 2.2 – Panoul de control Docker	29
Figura 2.3 – Diagrama Entitate-Relație (ERD) a platformei	32
Figura 2.4 – Arhitectura sistemului și infrastructura Docker	33
Figura 3.1 – Interceptarea și respingerea unei alocări conflictuale de către algoritm	36
Figura 3.2 – Log-ul modului de inițializare automatizată a datelor (seeding)	38
Figura 3.3 – Generarea dinamică a intervalelor orare și structura răspunsului JSON	43
Figura 3.4 – Testarea securității endpoint-urilor utilizând clientul API Postman.....	46
Figura 3.5 – Stocarea securizată a credențialelor utilizând algoritmul bcrypt	47
Figura 4.1 – Adaptabilitatea interfeței grafice implementată pe un viewport mobil	52
Figura 4.2 – Evitarea supraîncărcării DOM-ului prin randarea unui set de date redus	53
Figura 4.3 – Interfața componentei tabelare cu paginare gestionată de server	55
Figura 4.4 – Interceptarea erorilor de validare și injectarea dinamică a dicționarelor de traducere	57
Figura 4.5 – Interceptarea răspunsurilor de succes (HTTP 2xx) și gestionarea feedback-ului asincron ..	58
Figura 4.6 – Vizualizarea interactivă a agregărilor de stare și gestionarea stării componentelor grafice ..	60

Lista acronimelor

API	- Application Programming interface (Set de reguli și protocoale care permite microserviciilor sau aplicațiilor să comunice între ele)
ARIA	- Accessible Rich Internet Applications (Set de attribute HTML care fac aplicațiile web complexe mai accesibile persoanelor cu deficiențe)
BSON	- Binary JSON (Reprezentare binară a documentelor de tip JSON, formatul nativ utilizat de MongoDB pentru stocare eficientă)
CORS	- Cross-Origin Resource Sharing (Mecanism de securitate HTTP care permite sau blochează solicitările de resurse provenite de pe domenii/porturi diferite)
CRUD	- Create, Read, Update, Delete (cele patru operațiuni de bază necesare pentru gestionarea datelor persistente într-o bază de date)
CSRF	- Cross-Site Request Forgery (Atac malițios prin care comenzi neautorizate sunt transmise și executate în numele unui utilizator deja autentificat)
DOM	- Document Object Model (Interfață de programare care reprezintă documente web ca structuri de arbori cu noduri manipulabile)
DNS	- Domain Name System (Sistem de denumire ierarhic care traduce numele de domenii în adrese IP)
DRY	- Don't Repeat Yourself (Principiu fundamental de inginerie software ce vizează eliminarea repetiției logicii prin abstractizare)
ERD	- Entity-Relationship Diagram (Reprezentare vizuală arhitecturală a structurii unei baze de date, ilustrând entitățile și legăturile dintre acestea).
HTML	- HyperText Markup Language (Limbaj de marcare standard utilizat pentru structurarea conținutului paginilor web)
HTTP	- HyperText Transfer Protocol (Protocolul de bază al rețelei utilizat pentru transferul de date pe World Wide Web)
I/O	- Input/Output (Operațiuni de intrare/ieșire între un sistem de calcul și mediul exterior, cum ar fi interogările bazei de date)
JSON	- JavaScript Object Notation (Format standardizat, ușor de citit, utilizat pentru serializare și schimb de date)
JWT	- JSON Web Token (Standard deschis pentru transmiterea securizată a informațiilor între părți sub forma unui obiect JSON)
LAMP	- Linux, Apache, MySQL, PHP (Stivă tehnologică tradițională pentru dezvoltarea aplicațiilor web server-side)
MERN	- MongoDB, Express.js, React.js, Node.js (Stivă tehnologică full-stack bazată exclusiv pe JavaScript)
MVC	- Model-View-Controller (Model arhitectural software care separă logica aplicației în manipulare de date, interfață și control)
MUI	- Material UI (Bibliotecă de componente UI pentru React, care implementează strict standardele Google Material Design)

NoSQL - Not Only SQL (Sistem de gestionare al bazelor de date care utilizează modele de stocare flexibile, non-tabulare, precum documentele)

ODM - Object Data Modeling (Tehnică de programare care mapează modelele de obiecte din cod cu documentele din bazele de date NoSQL, specifică Mongoose)

RAM - Random Access Memory (Memorie volatilă cu acces aleator, utilizată ca spațiu de lucru principal al unui server sau sistem de calcul)

RBAC - Role-Based Access Control (Mecanism de securitate care restricționează accesul la sistem în funcție de rolul administrativ al utilizatorului)

RCE - Remote Code Execution (Vulnerabilitate critică de securitate care permite unui atacator să execute comenzi arbitrare pe o mașină de la distanță)

REST - Representational State Transfer (Stil arhitectural pentru sisteme web distribuite, care utilizează metode HTTP standardizate pentru operațiuni asupra datelor)

SOC - Separation of Concerns (Principiu de inginerie software care împarte un sistem în secțiuni distincte, fiecare având o responsabilitate unică)

SOP - Same-Origin Policy (Concept critic de securitate web formulat de browser, care limitează interacțiunea scripturilor cu resurse de pe alte origini)

SPA - Single Page Application (Aplicație web care interacționează cu utilizatorul rescriind dinamic pagina curentă, eliminând reîncărcările complete de pagină)

WCAG - Web Content Accessibility Guidelines (Set de directive internaționale pentru îmbunătățirea accesibilității conținutului web)

YAML - YAML Ain't Markup Language (Format de serializare a datelor lizibil de către om, utilizat frecvent pentru fișiere de configurare, precum `docker-compose`)

Lista de termeni preluați din engleză

- Bearer** - O schemă de autentificare HTTP în care posesorul ("purtătorul") token-ului criptografic primește accesul la rutele sau resursele protejate.
- Browser** - Aplicație software (client) utilizată pentru accesarea, interpretarea și randarea vizuală a documentelor web și a aplicațiilor SPA.
- Backend** - Infrastructura de server a unei aplicații, invizibilă pentru utilizator, responsabilă de logica de business, securitate și manipularea bazei de date.
- Callback** - O funcție transmisă ca argument unei alte funcții, concepută pentru a fi invocată ulterior, de obicei după finalizarea unei operațiuni asincronă (I/O).
- Cloud** - Server, aplicație sau date accesibile prin intermediul internetului, mai degrabă decât local, de pe dispozitivul utilizatorului.
- Endpoint** - Un punct specific de acces (URL) al unui API, definit pe server, la care un client trimite o solicitare HTTP pentru a interacționa cu datele.
- Enterprise** - Nivel arhitectural și operațional de întreprindere, caracterizat prin cerințe ridicate de scalabilitate, securitate robustă și disponibilitate.
- Framework** - Un cadru de lucru software ce oferă o arhitectură standardizată și o colecție de biblioteci predefinite pentru a facilita dezvoltarea aplicațiilor complexe.
- Frontend** - Partea unei aplicații software cu care interacționează direct utilizatorul, incluzând elementele vizuale, randarea DOM-ului și logica de validare a clientului.
- Hashing** - Procesul criptografic de transformare al unei parole sau a unui set de date într-un șir de date de lungime fixă, matematic ireversibil.
- Linting** - Procesul de analizare statică a codului sursă pentru a detecta erori de sintaxă, defecte sau deviații de la standardele de formatare (exemplu: ESLint).
- Log** - Jurnalul de activitate al unei aplicații/al unui serviciu, care confirmă starea de rulare a microserviciilor.
- Middleware** - Funcții software intermediare de pe server care interceptează, validează sau procesează cererile HTTP înainte ca acestea să atingă controller-ul final.
- Payload** - Sarcina utilă; corpul principal al unui pachet de date transmis (cum ar fi conținutul unui JWT sau datele dintr-o cerere HTTP POST) care conține informația esențială.
- Pipeline** - O serie liniară de elemente de procesare a datelor, unde ieșirea unei etape servește ca intrare pentru următoarea (utilizat frecvent în agregările MongoDB).
- Prop Drilling** - O anti-paradigmă în dezvoltarea componentelor (precum React) în care datele sunt transmise ineficient prin multiple straturi de componente intermediare care nu le utilizează.
- Salting** - Adăugarea unui set de date aleatorii (salt) la o parolă înainte de aplicarea funcției hash, pentru a contracara atacurile cu tabele precalculate (rainbow tables).
- Seeding** - Procesul de automatizare a populării inițiale a unei baze de date de producție cu valori predefinite sau date de test generate dinamic.
- Token** - Un jeton digital semnat criptografic, utilizat pentru a dovedi identitatea sau drepturile de acces ale unui utilizator în cadrul platformei.

- Thread - Fir de execuție; cea mai mică secvență de instrucțiuni programate care poate fi gestionată independent de planificatorul sistemului de operare.
- Viewport - Zona vizibilă, activă a unui ecran sau a unei ferestre software unde este afișat conținutul

Introducere

Introducere și context general

În peisajul educațional curent, digitalizarea fluxurilor de lucru administrative nu mai reprezintă un simplu lux operațional, ci o necesitate fundamentală pentru eficiența instituțională. Facultățile moderne funcționează ca ecosisteme deosebit de complexe, necesitând o gestionare continuă și sincronizată a mii de studenți, zeci de cadre didactice și a unei infrastructuri fizice aflată în dezvoltare.

În mod tradițional, administrația academică se bazează, deseori, pe sisteme fragmentate, pe monitorizarea prin tabele statice și pe introducerea manuală a datelor. Această abordare descentralizată introduce, inevitabil, erori umane, generând erori ale sistemului, cum ar fi dubla rezervare a sălilor de curs, suprapunerea diferitelor activități universitare, și la un timp de așteptare prelungit la biroul de secretariat. Când datele sunt fragmentate în diferite departamente diferite și izolate unul de celălalt, generarea unui mediu academic coerent și lipsit de conflicte devine o sarcină copleșitoare din punct de vedere computațional și logistic.

Motivația proiectului

Motivația principală din spatele acestui proiect este proiectarea și implementarea unei soluții software centralizate, capabilă să rezolve aceste blocaje administrative. Tranziția de la lucrul manual la o platformă web automatizată, nativă în cloud, elimină fricțiunea administrației învechite. Prin implementarea unui ecosistem digital unificat, platforma asigură că personalul administrativ, cadrele didactice și studenții au acces sincronizat, în timp real, la o singură sursă de adevăr, transformând astfel un proces administrativ reactiv într-un sistem proactiv și optimizat.

Obiective propuse și contribuții personale

Obiectivul principal al acestei lucrări este de a proiecta, dezvolta și containeriza o platformă full-stack complexă pentru administrația facultății. Contribuțiile personale la acest proiect înglobează întregul ciclu de viață al dezvoltării software, de la conceptualizarea arhitecturală până la implementarea finală de tip DevOps.

Obiectivele de bază atinse includ:

- **Programare algoritmică inteligentă:** Proiectarea unui algoritm personalizat, pe mai multe niveluri, de detectare a suprapunerilor, capabil să valideze orarele academice în timp real, prevenind matematic conflictele între sălile fizice, cadrele didactice și grupele de studenți.
- **Arhitectură decuplată și RBAC:** Implementarea unui API RESTful securizat utilizând JSON Web Tokens (JWT) pentru a impune un Control al Accesului Bazat pe Roluri (RBAC) strict, furnizând seturi de date și interfețe operaționale personalizate pentru administratori, profesori, studenți și reprezentanți externi.
- **Modelarea datelor non-relaționale:** Proiectarea structurală a unei scheme de baze de date MongoDB (NoSQL) extrem de optimizate, utilizând entități asociative pentru a simula integritatea relațională fără a sacrifica flexibilitatea documentelor.

- **Implementare UI/UX modernă:** Construirea unei Aplicații cu o Singură Pagină (SPA) reactivă și bazată pe stări folosind React.js și Material-UI, oferind interfețe accesibile și adaptabile oricărui dispozitiv.
- **Automatizarea infrastructurii:** Dezvoltarea de scripturi de populare automată a datelor pentru inițializarea dinamică a mediului, culminând cu containerizarea completă a sistemului prin Docker pentru o implementare independentă de platformă.

Structura lucrării

Pentru a oferi o imagine de ansamblu asupra dezvoltării sistemului, această lucrare este împărțită structural în patru capitole principale:

- Capitolul 1 prezintă o analiză critică a stivei tehnologice selectate, justificând implementarea ecosistemului MERN și explorând mecanica teoretică din spatele Node.js și a framework-urilor frontend moderne.
- Capitolul 2 detaliază arhitectura de nivel înalt a sistemului, concentrându-se pe rețeaua decuplată client-server, pe strategiile de proiectare a bazelor de date non-relaționale și structura de containerizare DevOps.
- Capitolul 3 aprofundează algoritmi de bază ai backend-ului, documentând riguros execuția matematică a matricei temporale de coliziune, măsurile de securitate criptografică și secvența de inițializare automată.
- Capitolul 4 conturează arhitectura vizuală a frontend-ului, explorând strategiile de gestionare a stărilor, mecanisme de rutare și integrarea sistemelor de design accesibile.

Capitolul 1. Tehnologii și Instrumentele Software

1.1. Arhitectura stivei MERN

Pentru a putea obține performanța și scalabilitatea necesare unei aplicații web moderne, proiectul a fost dezvoltat folosind stiva MERN. Această stivă tehnologică a fost aleasă datorită mediului său lingvistic unificat; JavaScript este utilizat atât pe partea de client (frontend), cât și pe partea de server (backend), permițând un flux de date JSON fără probleme de la baza de date direct către interfața folosită de utilizator.

1.1.1. Ecosistem lingvistic unificat, sau stive tradiționale

În timpul alegerii arhitecturii unei platforme web moderne, selecția stivei tehnologice determină nivelul de scalabilitate al sistemului și ciclul de viață al dezvoltării. Arhitecturile tradiționale, cum ar fi stiva LAMP sau mediul enterprise Java Spring, necesită schimbarea contextului între limbaje de programare complet diferite (exemplu: logica serverului este scrisă în PHP/Java și logica clientului este scrisă în JavaScript).

Stiva MERN a fost aleasă pentru a impune un ecosistem lingvistic unificat. Utilizând JavaScript (și, prin extensie, structuri de date JSON) în ciclul de viață al întregii aplicații (de la motorul bazei de date MongoDB, prin mediul de execuție al serverului Node.js și, în final, la browserul clientului React) înseamnă că sarcina de serializare și deserializare a datelor este scăzută semnificativ. Un payload primit de la baza de date poate fi consumat direct de componentele frontend fără necesitatea unui nivel de transformare al fluxului de date. Acest abordare izomorfă reduce complexitatea arhitecturală, reduce latența analizelor viitoare, și crește viteza de dezvoltare a noi funcționalități administrative.

1.2. Tehnologii Frontend

React.js

Interfața de utilizator a fost construită folosind React.js, o bibliotecă JavaScript open-source dezvoltată de Meta [1]. React permite crearea unei aplicații cu o singură pagină (SPA), ceea ce înseamnă că browserul încarcă un singur document HTML și actualizează dinamic conținutul, fără a fi nevoie de reîncărcări complete ale paginii. Acest lucru se realizează prin intermediul DOM-ului virtual din React, care calculează eficient modificări minime ale interfeței de utilizator, rezultând o experiență a utilizatorului extrem de receptivă. Gestionarea stărilor în cadrul aplicației (cum ar fi sesiunea utilizatorului activ, preferințele lingvistice și tema vizuală) a fost realizată folosind API-ul Context, încorporat în React.

Material-UI (MUI)

Pentru a asigura un design consistent, profesional și accesibil, a fost integrată biblioteca de componente Material-UI (MUI). Aceasta oferă componente robuste, predefinite (cum ar fi tabele de date, modale și panouri de navigare) bazate pe principiile Material Design ale Google. Acesta acceptă nativ

designul web receptiv, asigurând că platforma este utilizabilă atât pe desktop, cât și pe dispozitive mobile, și a facilitat implementarea comutatorului modului luminos/întunecat.

Axios

Comunicarea dintre frontend și API-ul backend a fost gestionată folosind Axios, un client HTTP bazat pe promisiuni. A fost implementat un interceptor Axios global pentru a atașa automat token-uri de autentificare la solicitări și pentru gestionarea expirării token-urilor în siguranță, redirectionând utilizatorii către portalul de autentificare.

1.2.1. DOM-ul virtual și reutilizarea de componente

Decizia de a utiliza React.js în locul altor framework-uri competitive (exemplu: Angular sau Vue.js) a fost influențată de cerințele necesare de performanță, întrucât vor fi randate tabele cu o multitudine de date dinamice (cum ar fi un orar universitar, sau interfața de gestionare a utilizatorilor).

Manipularea DOM tradițională este costisitoare din punct de vedere computațional. Atunci când un administrator sortează sau filtrează un tabel, forțează browserul să recalculeze și să construiască întregul document HTML. Această acțiune va provoca disconfort vizual utilizatorului. Această problemă este înlăturată de React datorită arhitecturii acestuia bazată pe DOM virtual. Când starea interfeței grafice se schimbă, React calculează exact diferența dintre interfața grafică actuală și cea cerută de utilizator în memorie, apoi aplicând doar actualizările necesare DOM-ului din browser.

În plus, arhitectura React bazată pe componente permite crearea elementelor de interfață grafică reutilizabile (exemplu: o componentă standard `<PaginatedTable>` care poate fi populată ori cu date ale utilizatorilor, ori cu date despre sălile din universitate). Spre deosebire de Angular, care impune o structură MVC, React oferă flexibilitatea de a se integra perfect cu API-ul decuplat Node.js fără a introduce resurse suplimentare și inutile pe partea de client.

1.2.2. Arhitectura rețelei și interceptoare HTTP

Deși API-urile browserelor native, precum `fetch`, oferă capabilități de bază pentru realizarea solicitărilor de rețea, aplicațiile de nivel enterprise necesită un control granular asupra ciclului de viață al solicitărilor și răspunsurilor HTTP. De aceea, Axios a fost selectat și implementat nu doar ca un simplu client HTTP, ci ca un nivel de gestionare al rețelei, în principal datorită arhitecturii sale bazate pe interceptoare.

Într-o platformă administrativă securizată, atașarea manuală a jetoanelor JWT la fiecare cerere individuală către server este redundantă și dispusă la erori umane. Pentru a automatiza acest proces, a fost configurat un interceptor global Axios la nivelul frontend-ului. Înainte ca orice cerere să părăsească browserul, acest interceptor oprește execuția, accesează memoria locală (și securizată) a clientului, extrage jetonul de autentificare și îl injectează dinamic în antetul Authorization sub forma unui jeton de tip „Bearer” [20].

Mai mult, interceptoarele de răspuns acționează ca o barieră globală. Dacă backend-ul returnează o eroare HTTP 401 (neautorizat), indicând astfel expirarea jetonului JWT, interceptorul captează eroarea înainte ca aceasta să ajungă la componentele React, distruge starea sesiunii curente și forțează o

redirecționare automată și sigură către portalul de autentificare, prevenind astfel expunerea accidentală a interfețelor restricționate.

1.2.3. Motoare de stilizare CSS-in-JS și standarde de Accesibilitate (WCAG)

Implementarea interfeței grafice pentru o instituție publică, cum este o facultate, necesită respectarea unor standarde stricte de accesibilitate web, asigurând că platforma poate să fie utilizată și de către persoanele cu deficiențe de vedere sau motorii. MUI a fost integrat tocmai pentru a satisface aceste cerințe arhitecturale, fiind construit în conformitate cu normele WCAG [19]. Toate componentele predefinite gestionează automat navigarea prin tastatură și attributele ARIA.

Din punct de vedere al ingineriei software frontend, MUI rezolvă problema complexității CSS-ului global prin utilizarea paradigmei CSS-in-JS, susținută de motorul Emotion [21]. În loc să se bazeze pe fișiere .css externe greu de întreținut, stilizarea este injectată direct în componentele React la momentul execuției. Acest motor calculează și generează clase CSS unice doar pentru componentele aflate activ în DOM-ul virtual.

Această arhitectură elimină complet coliziunile de nume între clasele CSS și reduce drastic dimensiunea fișierelor descărcate de browser la momentul încărcării aplicației. De asemenea, facilitează manipularea dinamică a contextului vizual, permițând comutarea instantanee între modurile luminos și întunecat prin injectarea unor variabile de temă globale direct în arborele de componente.

1.3. Tehnologii Backend

Node.js și Express.js

Infrastructura backend este susținută de Node.js, un mediu de execuție JavaScript construit pe motorul V8 al Chrome [3], cunoscut pentru arhitectura sa neblokantă, bazată pe evenimente. Pe lângă Node.js, framework-ul Express.js a fost utilizat pentru a construi un API RESTful. Express.js a furnizat mecanismele de rutare și middleware-ul necesare pentru a gestiona cererile HTTP, a valida datele de intrare și a aplica protocoalele de securitate înainte de a interacționa cu baza de date.

JSON Web Tokens (JWT)

Autentificarea și autorizarea au fost securizate folosind JSON Web Tokens (JWT) [4]. După o autentificare realizată cu succes, serverul emite un token semnat digital care conține identitatea și rolul utilizatorului (desigur, criptate). Acest token este folosit ulterior pentru a acorda sau a refuza accesul la endpoint-uri administrative protejate, asigurându-se că, spre exemplu, doar utilizatorii cu rol de administrator pot executa operațiuni CRUD (Create, Read, Update, Delete) asupra entităților/modelelor Sală sau Curs.

1.3.1. I/O asincron și arhitectura buclei de evenimente

Serverele web tradiționale (exemplu: Apache) funcționează pe un model cerere-răspuns cu mai multi-thread, în care fiecare solicitare HTTP primită generează un fir de execuție nou și dedicat. Cu toate că acest model pare simplu la nivel de concept, va deveni extrem de ineficient la scară mai mare, deoarece

cererile concurente consumă rapid memoria RAM a serverului și implică o supraîncărcare severă cauzată de comutarea contextului.

Node.js evită această limitare hardware utilizând o arhitectură single-thread, cu un grad ridicat de concurență, condusă de bucla de evenimente și de biblioteca C++ `libuv` [16]. Atunci când serverul Express.js primește simultan mai multe solicitări (exemplu: mai mulți studenți încearcă să-și facă programări la secretariat în același timp), Node.js nu blochează firul principal de execuție pentru a aștepta răspunsul bazei de date MongoDB. În schimb, trimite aceste interogări de intrare/ieșire (I/O) către un grup de procese de lucru din nucleu sistemului.

Odată ce baza de date finalizează operațiunea, o funcție de tip callback este plasată în coada de evenimente. Bucla de evenimente monitorizează continuu această coadă, împingând callback-urile rezolvate înapoi în stiva de apeluri principală pentru execuție. Această arhitectură asincronă și non blocantă asigură faptul că API-ul administrativ rămâne extrem de receptiv și de eficient din punct de vedere al memoriei, fiind capabil să gestioneze mii de conexiuni concurente pe un server standard, fără a epuiza firele de execuție.

1.3.2. Structura criptografică și autentificarea fără stare

Autentificarea tradițională, bazată pe sesiuni, impune serverului să stocheze un ID de sesiune în memoria sa pentru fiecare utilizator conectat. Într-un mediu SPA modern și decuplat, această abordare care menține starea introduce probleme mari de scalabilitate. JWT rezolvă această problemă mutând starea pe partea de client, utilizând astfel un protocol fără stare, verificabil matematic [17].

Un JWT standard este compus din trei segmente distincte, separate prin puncte și codificate folosind formatul Base64Url: antet, payload și semnătura.

- Antetul definește algoritmul criptografic utilizat (în această platformă, `HS256` sau HMAC SHA-256) și tipul jetonului.
- Payload-ul conține revendicările, adică datele efective care afirmă identitatea utilizatorului, cum ar fi `objectId`-ul lor din baza de date, rolul lor administrativ (`role`) și un marcaj temporal pentru expirare (`exp`).
- Semnătura este mecanismul critic de securitate. Aceasta este generată preluând antetul și payload-ul, ambele codificate, și aplicându-le o funcție de hashing, împreună cu o cheie secretă protejată (`JWT_SECRET`), stocată exclusiv în variabilele de mediu ale serverului.

Deoarece frontend-ul React conține doar jetoane codificate, dar îi lipsește cheia secretă, orice încercare a unui atacator de a altera payload-ul (exemplu: să își schimbe rolul din student în administrator) va invalida automat semnătura. Când middleware-ul Express.js interceptează jetonul manipulat, verificarea hash-ului criptografic va eșua, respingând proactiv solicitarea frauduloasă. Acest mecanism asigură că platforma poate avea încredere în credențialele furnizate de client, fără a mai interoga baza de date pentru fiecare solicitare HTTP în parte.

1.4. Bază de date și modelarea datelor

MongoDB și Mongoose

Considerând natura dinamică și puternic interconectată a datelor platformes (de exemplu, o intrare care leagă o sală, un curs și un profesor), a fost selectată o bază de date non-relațională (NoSQL). MongoDB stochează datele în documente flexibile, de tip JSON [2], devenind astfel o alegere naturală pentru o aplicație bazată pe JavaScript.

Pentru a putea structura și valida în cadrul acestui mediu non-relațional, Mongoose a fost folosit, pentru a putea defini strict schemele de date (specificând câmpuri obligatorii, tipuri de date și valori implicite), și a facilita crearea de relații între diferite modele de date, folosind sistemul de referință ObjectId.

1.4.1. Necesitatea unei scheme flexibile

Într-un sistem administrativ universal tradițional, datele sunt, de obicei, stocate în baze de date relaționale SQL (exemplu: PostgreSQL sau Oracle). În timp ce SQL excelează în consistența tranzacțională strictă, este în mod inerent inflexibil. Încercarea de a modifica o schemă relațională pentru a adăuga un nou tip de utilizator sau de a adăuga vectori dinamici de echipamente disponibile într-o sală necesită migrări complexe de baze de date (ALTER TABLE), acțiune care va rezulta în timp de nefuncționare a platformei.

Dat fiind faptul că o platformă academică administrativă va evolua în timp, a bază de date bazată pe documente non-relaționale (NoSQL) a fost aleasă pentru a profita la maxim de flexibilitatea schemelor. MongoDB stochează datele ca documente BSON [18]. Acest polimorfism permite colecției User să stocheze tipuri de date fundamental diferite în același timp. Astfel, un document pentru studenți poate conține un obiect complex pentru detalierea anului de studiu, grupei din care face parte și al specializării pe care acesta a ales-o, iar un document pentru administratori va omite complet acest câmp fără a încălca constrângerile tabelului sau a încărca tabela cu valori nule. Mongoose a fost ulterior integrat pentru a furniza un nivel în plus de validare, la nivel de aplicație, asigurând astfel că, în timp ce baza de date rămâne la fel de flexibilă, logica aplicației încă impune regulile esențiale de integritate a datelor.

1.5. Instrumente de dezvoltare și implementare

- **Visual Studio Code:** Folosit ca principal mediu de dezvoltare integrat (IDE), datorită extensiilor oferite (formatare cod, linting – analizarea codului, fără executarea programului, pentru a detecta posibile erori, evidențiere sintaxă).
- **Docker:** Utilizat pentru containerizarea aplicației. Prin încapsularea bazei de date, a backend-ului și a frontend-ului în containere Docker izolate, se asigură că proiectul poate fi implementat și executat pe orice altă mașină, fără conflicte de dependențe.
- **MongoDB Compass:** Utilizat ca interfață grafică pentru interogarea, gestionarea și vizualizarea bazei de date NoSQL, facilitând monitorizarea colecțiilor de documente și validarea pipeline-urilor de agregare pe parcursul dezvoltării

Capitolul 2. Arhitectura sistemului si modelarea datelor

2.1. Arhitectură client-server decuplată

Platforma este gândită și proiectată folosind o arhitectură strictă, și anume decuplarea client-server. În acest fel, este asigurată separarea responsabilităților (SoC). Cum se poate observa în structura proiectului, aplicația este împărțită în două medii principale:

- Server (backend): Cuprinde întreaga logica operațională de bază, routarea API, partea de middleware (cum ar fi validarea JWT), și schemele bazei de date.
- Client (frontend): Se ocupă de partea vizuală, componentele interfeței grafice (UI) și de gestionarea stărilor globale.

Această metodă de separare permite ambelor părți ale aplicației să fie scalate și întreținute mai ușor, și, potențial, să fie folosite/implementate independent.

2.1.1. Protocolul de comunicare RESTful API

Pentru a ușura comunicarea între arhitectura decuplată client și server, sistemul implementează un API bazat pe arhitectura REST. Serverul Node.js expune o serie de puncte de acces corespunzătoare operațiilor standard CRUD.

Clientul React inițiază solicitări asincrone HTTP către aceste puncte de acces folosind clientul Axios, folosind strict verbe HTTP: solicitările `GET` sunt folosite exclusiv pentru obținere de date (exemplu: preluarea orarului academic), solicitările `POST` sunt folosite pentru a trimite noi payload-uri (exemplu: rezervarea unei săli), solicitările `PUT` sunt folosite pentru modificări de stare (exemplu: aprobarea unei solicitări aflate în așteptare), iar solicitările `DELETE` sunt folosite pentru distrugerea înregistrărilor. Toate schimburile de date sunt realizate în format JSON, asigurând astfel o comunicare ușoară, standardizată și independentă de limba vorbită între interfața vizuală și motorul bazei de date.

2.2. Controlul accesului bazat pe roluri (RBAC – Role-Based Access Control)

Logica acestei aplicații se bazează pe schema `User`, care dictează sistemul controlului accesului bazat pe roluri. Un `enum` (un set/șir de constante) impune 4 roluri distincte: `admin`, `teacher`, `student` și `external_representative`. Pentru a menține o structură unificată, dar flexibilă și non-relațională, atributele specifice per rol (cum ar fi: `yearOfStudy`, `specialization` și `group`) sunt stocate în aceeași colecție, dar sunt procesate condițional, bazat pe rolul utilizatorului.

2.2.1. Securitatea rutelor cu ajutorul Middleware cu JWT

Deși schema bazei de date definește utilizatori cu diferite roluri, aplicarea efectivă a securității platformei este gestionată la nivel de rețea prin funcțiile de middleware ale Express.js. Sistemul RBAC nu se bazează doar pe ascunderea elementelor vizuale în interfața utilizatorului, el respinge activ solicitări neautorizate la nivel de server.

Când un utilizator se autentifică, serverul generează un JWT semnat criptografic care încapsulează ID-ul lor din baza de date și rolul administrativ pe care îl au în platformă. Pentru fiecare solicitare următoare către o rută protejată (exemplu: modificarea entităților `Room`), middleware-ul extrage acest jeton din

antetul `HTTP Authorization`. Apoi, serverul decodează jetonul folosind o cheie privată (`JWT_SECRET`) pentru a se asigura că payload-ul nu a fost manipulat în tranzit. Dacă rolul decodat este cel de administrator, middleware-ul apelează următoarea funcție `next()` pentru a permite execuția controlerului. Altfel, procesul este întrerupt instant, iar serverul returnează codul de stare `HTTP 403 (interzis)`, asigurând astfel o protecție impenetrabilă împotriva escaladării verticale a privilegiilor.

2.3. Strategii relaționale într-un mediu non-relațional

Deși MongoDB este o bază de date non-relațională, o platformă de administrare universitară necesită date interconectate. Pentru a preveni duplicarea datelor și a păstra uniformitate, sistemul de referință `ObjectId` al Mongoose, și anume `ref`, este aplicat în toate schemele create.

Infrastructura academică

Modelul `Course` este proiectat să gestioneze repartizări complexe ale cadrelor didactice. În loc să stocheze text, acesta folosește referințe ale `ObjectId` pentru a asigura profesori distincți (`lectureProfessor`, `seminarProfessor`, `labProfessor`) unui singur curs, gestionând astfel logistica unei universități. Similar, modelul `Room` cuprinde metadate structurale, acestea incluzând capacitatea totală a unei săli și un vector de echipamente disponibile în respectiva sală. În acest mod, reprezentanții externi vor avea toate informațiile necesare pentru a putea rezerva o sală.

Baza Orarului

Piesa centrală a aplicației este schema `ScheduleEntry`. Aceasta funcționează ca o joncțiune relațională, conectând dinamic un curs (`Course`), o sală (`Room`), și un utilizator (`User`) (mai exact profesorul/profesorii care predau acea materie), la o zi a săptămânii (`dayOfWeek`), un interval orar (`timeSlot`) și la o grupă de studenți (`group`). Această structură normată facilitează algoritmul avansat de detecție a suprapunerilor, întrucât permite bazei de date să facă referințe către alte entități.

Proceduri administrative

Activitățile zilnice sunt definite în schemele `Appointment` și `RoomReservation`. Ambele proceduri folosesc un `enum` de `status` (`pending`, `approved`, `rejected`, `completed`) pentru a monitoriza ciclul de viață al unui request, menținând astfel integritatea referințelor prin conectarea cererii înapoi la utilizator și la resursa solicitată.

2.3.1. Integritate referențială sau încorporarea datelor

Utilizând MongoDB, programatorii sunt nevoiți să aleagă între două mari strategii de modelare a datelor: încorporarea datelor direct în documentul părinte, sau utilizând referințe către documente externe prin intermediul unicilor `ObjectIDs`. Având în vedere natura interconectată a datelor unei universități, încorporarea datelor ar duce la o redundanță inutilă și la anomalii de sincronizare. De exemplu, dacă numele unui profesor ar fi schimbat, actualizarea unui sistem încorporat ar consta în verificarea și modificarea a mii de documente `ScheduleEntry` mai vechi.

Pentru a impune normalizarea datelor într-un mediu NoSQL, arhitectura folosește exclusiv referințe către documente. Modelul `ScheduleEntry` funcționează ca o entitate asociativă (similar cu o tabelă de joncțiune pentru bazele de date relaționale), stocând doar `ObjectId`-ul de 24 de caractere, hexazecimal, al entităților asociate `Room`, `Course` și `User`. Astfel, este garantată consistență absolută: modificările aduse capacității unei săli sau a numelui unui curs sunt instant reflectate în toate rezervările active, fără a fi nevoie de actualizări în cascadă.

2.3.2. Joncțiuni cu ajutorul metodei `populate()` a `Mongoose`

Deoarece MongoDB nu suportă declarații `JOIN` (nativ SQL) în limbaj standard de interogări, crearea acestor relații necesită anumite intervenții în arhitectura aplicației. Pentru operații de simple, de obținere de date dintr-un singur document (exemplu: aflarea de detalii despre o singură programare), serverul utilizează metoda `populate()` a `Mongoose`.

Când este apelată, librăria `Mongoose ODM` interceptează interogarea, citește documentul, extrage referințele `ObjectId` salvate, iar apoi execută automat interogări secundare pentru a obține documentele complet hidratate (exemplu: obținerea adresei de e-mail și a numelui de utilizator din colecția `User`). Aceste documente sunt apoi injectate dinamic în payload-ul de răspuns JSON din memoria Node.js, simulând astfel o joncțiune relațională la nivel de aplicație.

2.3.3. Agregări relaționale complexe folosind operatorul `$lookup`

Chiar dacă metoda `populate()` este eficientă pentru interogări separate, aceasta va deveni inevitabil costisitoare din punct de vedere computațional pentru operații de recuperare a datelor în masă, cum ar fi încărcarea orarului universitar paginat. Preluarea a sute de intrări ale unui orar academic și executarea ulterioară a unor interogări secundare individuale pentru fiecare sală și fiecare curs ar bloca bucla de evenimente Node.js.

Pentru a ajunge la nivelul de înalt de performanță cu aceste interogări, sistemul ocolește nivelul de aplicație și asignează acest volum de lucru direct motorului MongoDB folosind Pipeline-ul de agregare, în mod specific operatorul `$lookup`. Acest operator funcționează ca o joncțiune exterioară de stânga, executată direct în serverul bazei de date:

```
// Relational Aggregation Pipeline for the Academic Schedule
const pipeline = [
  { $match: filterQuery }, // Initial dataset reduction
  {
    $lookup: {
      from: 'courses', // The target collection
      localField: 'course', // The ObjectId stored in ScheduleEntry
      foreignField: '_id', // The primary key in the Course collection
      as: 'course' // The output array field
    }
  },
  {
```

```

    $lookup: {
      from: 'rooms',
      localField: 'room',
      foreignField: '_id',
      as: 'room'
    },
    // Deconstructs the resulting arrays back into plain object fields
    { $unwind: { path: '$course', preserveNullAndEmptyArrays: true } },
    { $unwind: { path: '$room', preserveNullAndEmptyArrays: true } }
  ];

```

Folosind operatorul `$lookup` împreună cu operatorul `$unwind`, serverul comprimă relațiile complexe dintre mai multe colecții într-un singur payload JSON unificat înainte de a-l trimite în rețea. Această decizie arhitecturală optimizează consumul de memorie, scade latența rețelei, și asigură faptul că platforma este extrem de scalabilă, și pregătită să acomodeze un volum mult mai mare de date.

Figura 2.1 demonstrează vizual procesul de interogare relațională direct la nivelul motorului NoSQL, utilizând MongoDB Compass. Această captură de ecran evidențiază flexibilitatea arhitecturii documentare: deși referințele prin `ObjectId` sunt frecvent utilizate, sistemul poate executa joncțiuni eficiente bazate pe potrivirea șirurilor de caractere. În panoul din stânga se observă datele de intrare ale cursului, în timp ce stadiul `$lookup` identifică automat contul profesorului corespunzător (`localField: 'professors.lecture'`). Panoul din dreapta confirmă injectarea cu succes a detaliilor complete ale utilizatorului în documentul de ieșire. Această abordare transferă sarcina computațională de la serverul Node.js către baza de date, optimizând masiv performanța aplicației la citire.

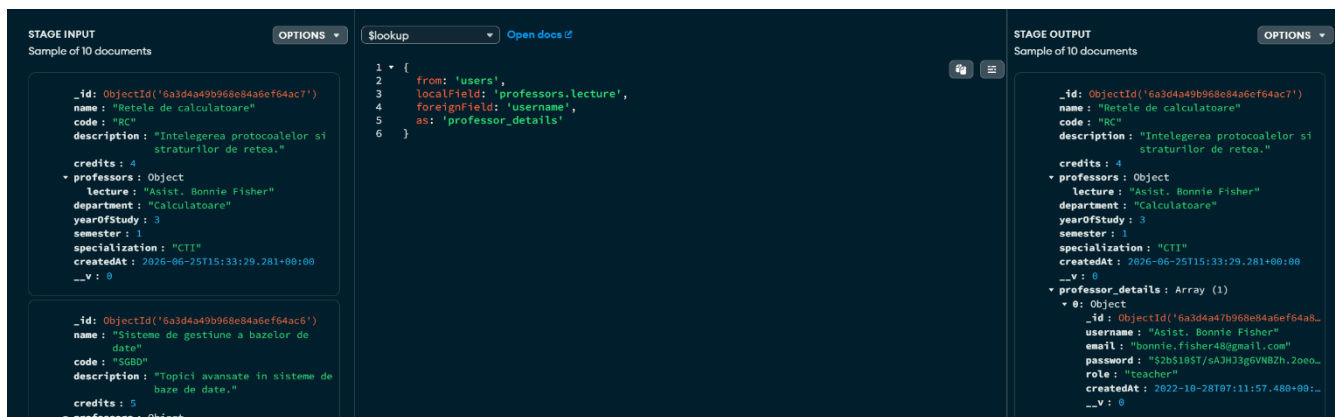


Figura 2.1 – Execuția stadiului de agregare `$lookup` pentru asamblarea datelor relaționale

2.4. Securitatea infrastructurii și containerizarea (DevOps)

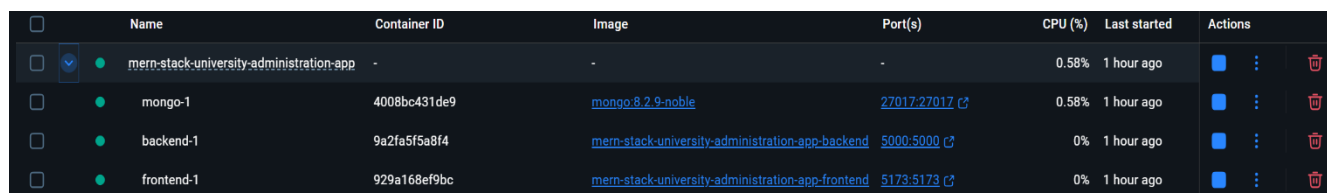
2.4.1. Orchestrarea microserviciilor folosind Docker

Aplicațiile web moderne necesită medii de implementare consistente pentru a prevenii anomaliile cauzate de componentele hardware ale gazdei sau de sistemul de operare existent (clasica scuză folosită

în implementarea software: „it works on my machine”). Pentru a standardiza fazele de implementare și de producție, platforma administrativă a fost containerizată folosind ecosistemul Docker [15].

Arhitectura utilizează comanda `docker-compose` pentru a orchestra simultan mediile izolate ale platformei. Fișierul de configurare YAML mapează porturile de rețea (exemplu: portul 5000 pentru server și portul 5173 pentru client) și declară o rețea virtuală internă. Această orchestrare garantează că serviciile MERN pot comunica între ele prin rezolvarea numelui containerului DNS (exemplu: serverul se conectează direct la container-ul `mongo`), în timp ce izolează fizic baza de date de expunerea directă la internet, adăugând astfel un nivel principal de securitate la nivel de infrastructură.

Figura 2.2 oferă o perspectivă asupra instanțelor containerizate gestionate prin ecosistemul Docker. Această imagine reflectă gestionarea coordonată a bazei de date (MongoDB), a serverului de aplicație (backend) și a interfeței client (frontend), confirmând succesul mapării porturilor și a rețelei virtuale interne menționate anterior. Această figură confirmă faptul că fiecare serviciu rulează într-un mediu izolat și stabil, demonstrând implementarea practică a strategiei de containerizare pentru eliminarea problemelor de compatibilitate și asigurarea unei infrastructuri portabile, conforme cu cerințele de securitate prin izolare logică a mediilor de execuție.



Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
mern-stack-university-administration-app	-	-	-	0.58%	1 hour ago	[Stop] [Restart] [Delete]
mongo-1	4008bc431de9	mongo:8.2.9-noble	27017:27017	0.58%	1 hour ago	[Stop] [Restart] [Delete]
backend-1	9a2fa5f5a8f4	mern-stack-university-administration-app-backend	5000:5000	0%	1 hour ago	[Stop] [Restart] [Delete]
frontend-1	929a168ef9bc	mern-stack-university-administration-app-frontend	5173:5173	0%	1 hour ago	[Stop] [Restart] [Delete]

Figura 2.2 – Panoul de control Docker

2.4.2. Securitatea containerului și principiul privilegiilor minime

O vulnerabilitate severă în implementările standard Docker este rularea aplicațiilor web cu privilegii de utilizator `root`. Dacă un atacator reușește să compromită o aplicație Node.js care funcționează cu privilegii `root`, acesta poate, teoretic, să dobândească acces total al întregului sistem gazdă.

Pentru a preveni această vulnerabilitate, o arhitectură cu securitate pe multiple etape a fost implementată în configurațiile fișierului `Dockerfile`. În etapa de producție, scriptul abandonează utilizatorul `root` și generează un grup dedicat (`nodejs`) și un utilizator izolat, non-root (`nodeuser` cu identificatorul de sistem 1001):

```
# Create a non-root user and group for security compliance
RUN addgroup -g 1001 nodejs && \
    adduser -u 1001 -G nodejs -S -D nodeuser

# Transfer directory ownership to the newly isolated user
RUN chown -R nodeuser:nodejs /app

# Switch the execution context to the non-root user
USER nodeuser
```

Aplicând principiul privilegiilor minime la nivel de containerizare, sistemul limitează pe cât de mult posibil potențialele daune venite în cazul unui potențial vulnerabilități RCE, aducând platforma la standarde de implementare de nivel enterprise.

2.5. Gestionarea stărilor globale și structura componentelor

2.5.1. Limitarea transmiterii datelor prin componente intermediare

Într-un framework bazat pe componente grafice așa cum este React, datele sunt trimise de sus în jos, de la părinte la copil prin intermediul componentelor `props`. În timp ce această metodă este eficientă pentru componente mici, devine rapid o limitare arhitecturală pentru o platformă administrativă complexă. Trimiterea jetoanelor de autentificare, a rolurilor utilizatorilor, sau a preferințelor grafice printr-un număr mare de niveluri de componente intermediare (care nu necesită aceste informații), rezultă într-o practică defectuoasă cunoscută sub numele de „prop drilling”. Această practică leagă structura aplicației și îngreunează mentenabilitatea codului.

2.5.2. Implementarea API-ului Context

Pentru a separa nivelul de date din ierarhia vizuală, arhitectura implementează API-ul Context, nativ React. Acest sistem funcționează ca un gestionator centralizat și global de stări. Cum poate fi observat și în structura principală a aplicației, componentele dedicate `<AuthProvider>` și `<ColorModeProvider>` împachetează matricea de rutare.

Această decizie arhitecturală asigură că stările critice ale aplicației (exemplu: sesiunea activă a utilizatorului, sau preferința acestuia pentru tema luminoasă/întunecată a interfeței grafice) sunt gestionate într-un singur loc, în cel mai înalt nivel al DOM-ului virtual. Astfel, orice componentă încorporată în cod (exemplu: o celulă din tabela `ScheduleManagement`) poate consuma direct aceste stări globale folosind interceptorul `useContext`, trecând astfel peste ierarhia componentelor intermediare. Această abordare modulară garantează că actualizările interfeței utilizatorului sunt instantanee, și perfect sincronizate în toată aplicația SPA.

2.6. Securitate la nivel de aplicație și pipeline-uri de middleware

2.6.1. Arhitectura CORS

Deoarece platforma utilizează o arhitectură decuplată, clientul React și serverul Node.js funcționează ca și entități separate în rețea, funcționând pe porturi distincte în timpul implementării (exemplu: portul 5173 pentru Vite și portul 5000 pentru Express), sau ca și containere izolate în producție. În mod implicit, un browser web modern impune o politică SOP, un mecanism de securitate critic care blochează solicitări HTTP asincrone între diferite origini pentru a preveni atacuri CSRF.

Pentru a facilita comunicarea între micro serviciile decuplate, arhitectura serverului configurează un pipeline CORS în middleware. Acest sistem definește explicit ce domenii externe și anteturi HTTP au voie să interacționeze cu API-ul. Injectând anteturi specifice (`Access-Control-Allow-Origin`) în

interiorul răspunsurilor HTTP ale serverului, API-ul crează un canal de comunicație securizat special pentru clienții validați ai React, blocând astfel domenii neautorizate din a executa apeluri API.

2.6.2. Ciclul de viață al solicitărilor middleware Express

În plus față de configurația CORS, serverul Express.js se comportă ca un pipeline secvențial de date bine structurat. Înainte ca orice solicitare HTTP să ajungă la un controler al bazei de date, trebuie să treacă o serie de funcții middleware la nivel de aplicație.

Această arhitectură utilizează analizatori nativi de payload-uri (exemplu: `express.json()`) pentru a intercepta fluxul de date HTTP nealterat și a-l transforma în obiecte JSON lizibile. Acest proces garantează că, în momentul în care un payload ajunge la o anumită funcție de rutare (exemplu: funcția `createAppointment`), informația primită a fost deja formatată, autentificată cu ajutorul JWT, și validată din punct de vedere structural. În acest mod, este garantat că date modificate sau rău intenționate nu vor interacționa direct cu motorul bazei de date.

2.7. Diagrama Entitate-Relație (ERD) și structura datelor

Pentru a vizualiza logica relațională implementată în mediul NoSQL, Figura 2.3 ilustrează Diagrama Entitate-Relație a platformei. Această schemă evidențiază fluxul de referințe `ObjectId` care menține integritatea sistemului. Modelul `User` acționează ca piesă principală pentru interacțiunile umane (principale într-o platformă administrativă), fiind direct legat de programările la secretariat și de rezervările individuale de săli.

De asemenea, diagrama evidențiază rolul colecției `ScheduleEntry`, care funcționează ca un tabel de joncțiune. Aceasta leagă structural entitățile distincte `Course` și `Room`, permițând astfel algoritmului de suprapunere să interogheze simultan sălile, capacitățile acestora și programul profesorilor, evitând redundanța datelor, cauzată de o posibilă duplicare a datelor în interiorul documentelor.

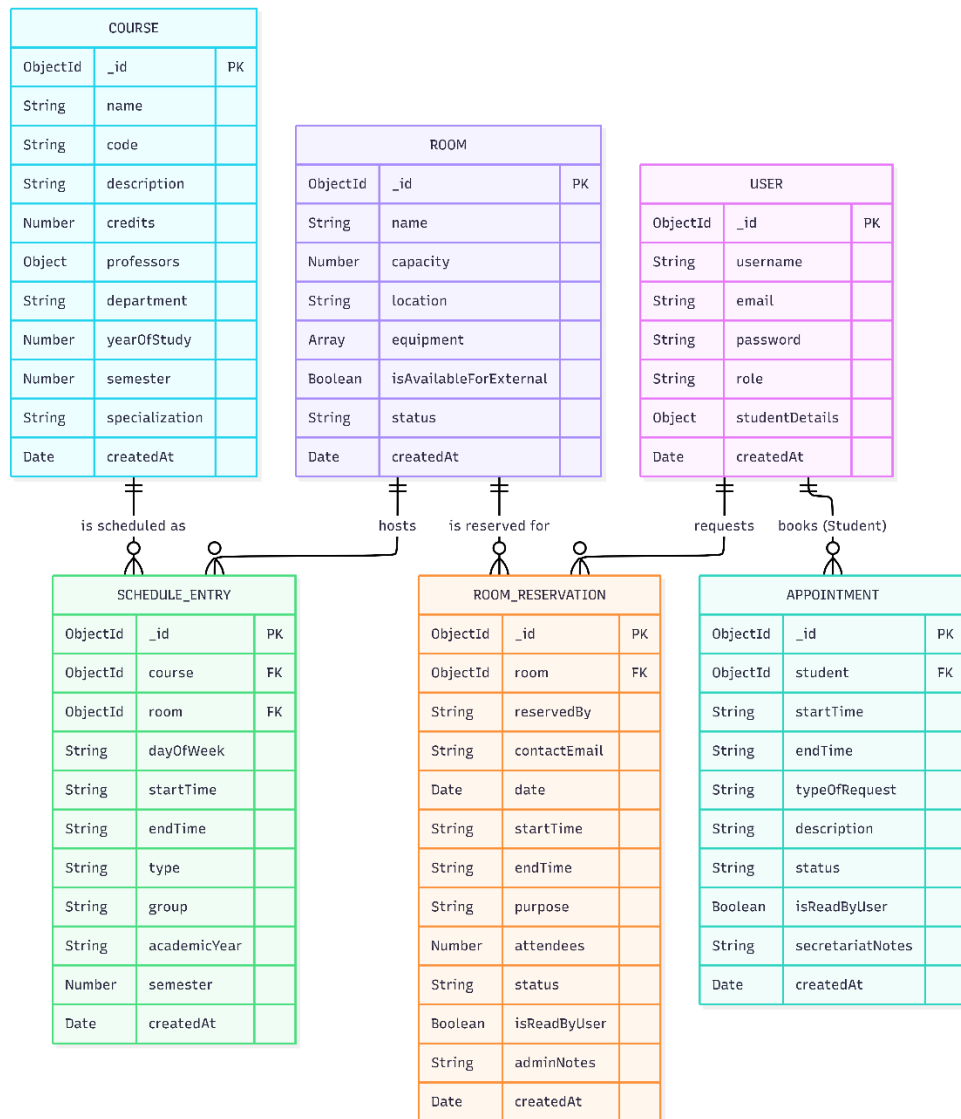


Figura 2.3 – Diagrama Entitate-Relație (ERD) a platformei

2.8. Fluxul arhitectural și topologia Sistemului

În figura 2.4 este reprezentată arhitectura completă a sistemului web, ilustrând ciclul de viață al unei solicitări, pornind de la interacțiunea utilizatorului, până la persistența din baza de date. Totul este, desigur, încapsulat în infrastructura Docker.

Când un utilizator interacționează cu interfața grafică (gestionată de Material UI și React Context), clientul HTTP Axios preia starea și construiește un payload JSON. Această solicitare traversează rețeaua internă Docker către containerul serverului, unde este, mai întâi, interceptată de middleware-ul de securitate (Express.js și JWT). Doar după confirmarea autorizației, pachetul de date este trimis către controlere și funcțiile de calcul temporal. La final, serverul execută interogările Mongoose optimizate pe containerul izolat MongoDB, returnând rezultatul pe același traseu securizat. Această topologie modulară asigură atât scalabilitatea aplicației, cât și izolarea completă a datelor sensibile față de partea publică a internetului.

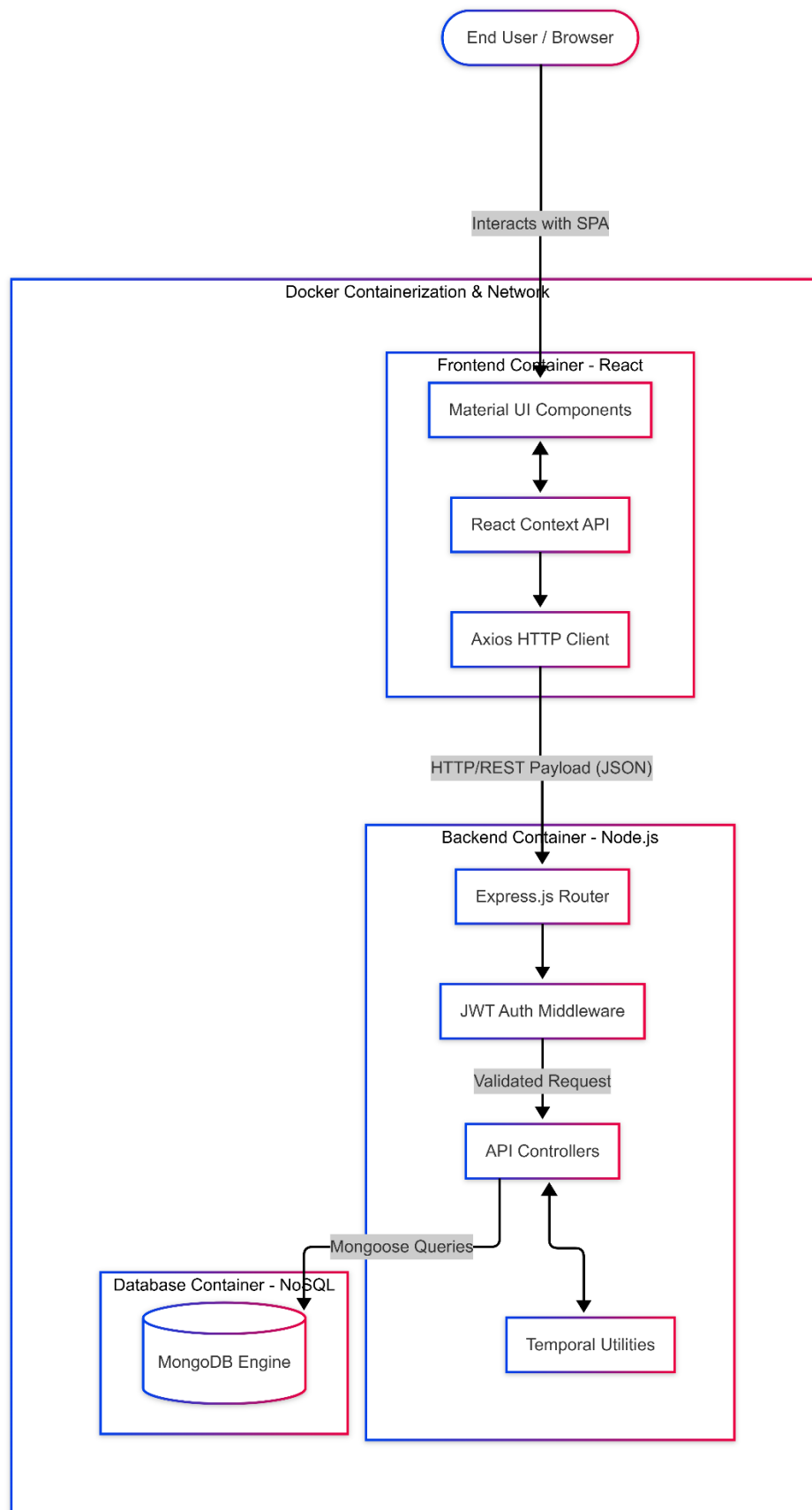


Figura 2.4 – Arhitectura sistemului și infrastructura Docker

Capitolul 3. Implementarea Algoritmilor de bază

3.1. Algoritmul de detecție a suprapunerilor pe 4 niveluri

Cel mai important mecanism al server-ului este algoritmul de validare al programărilor, care este proiectat pentru a face suprapunerile de orar imposibile matematic. Pentru a verifica dacă două cursuri sunt programate simultan, aplicația folosește o funcție matematică specifică. Astfel, în loc să compare formaturi de text complex (de exemplu, „14:30”), sistemul convertește ora de început și ora de sfârșit a unui curs în numărul total de minute. Apoi, compară aceste numere, pentru a vedea dacă intervalele orare sunt suprapuse. Logica din fișierul `scheduleController.js` este prezentată mai jos:

```
// Function to verify if two schedule entries happen at the same time
const checkOverlap = (existingEntry, newEntry) => {
  // If the days are different, there is no conflict
  if (existingEntry.dayOfWeek !== newEntry.dayOfWeek) {
    return false;
  }

  // Convert „HH:MM” string to total minutes for easy comparison
  const parseTime = (timeStr) => {
    const [hours, minutes] = timeStr.split(':', '').map(Number);
    return hours * 60 + minutes;
  };

  const existingStart = parseTime(existingEntry.startTime);
  const existingEnd = parseTime(existingEntry.endTime);
  const newStart = parseTime(newEntry.startTime);
  const newEnd = parseTime(newEntry.endTime);

  // Check if the time intervals intersect
  return newStart < existingEnd && newEnd > existingStart;
};
```

Pentru a transforma această verificare a suprapunerilor într-o formulă eficientă a bazei de date, server-ul se bazează pe operatori de interogare ai MongoDB. Algoritmul de validare pe patru niveluri construiește dinamic interogări relaționale utilizând operatorii `$or` și `$in` pentru a putea soluționa constrângerile pedagogice, precum unirea tuturor materiilor predate de un anume profesor, incluzând diferitele tipuri de activități (laborator, seminar, curs). În plus, un caz limită apare în timpul unei operații de actualizare, unde o intrare din orar poate semnală un conflict temporal fals, bazat pe fosta sa stare. Pentru a rezolva această situație, controller-ul exclude dinamic această intrare din matricea de verificare a suprapunerilor aplicând operatorul `$ne` (Not equal) asupra cheii sale primare (`_id`). Astfel, starea poate fi schimbată fără a declanșa erori.

Înainte de a salva o nouă clasă în baza de date, server-ul folosește această funcție (`checkOverlap`) pentru a verifica 4 elemente diferite (de unde și verificarea pe patru niveluri):

1. Sala: Verifică dacă sala este rezervată deja.
2. Profesorul: Verifică dacă profesorul/profesorii (de curs, de seminar, de laborator) au deja în orar un alt curs.
3. Grupa de studenți: Verifică dacă studenții din aceeași grupă nu au două activități în același timp.
4. Cursul: Previne adăugarea aceluiași curs/seminar/laborator de două ori pentru aceeași grupă de studenți.

Pentru a demonstra determinismul algoritmului de suprapunere, Figura 3.1 ilustrează răspunsul serverului la o încercare de dublă rezervare. Se observă că funcția de validare oprește execuția cererii HTTP înainte de interacțiunea cu baza de date, returnând un cod de eroare corespunzător și un mesaj descriptiv. Această blocare dovedește funcționarea corectă a filtrelor temporale și spațiale aplicate la nivelul controller-ului.

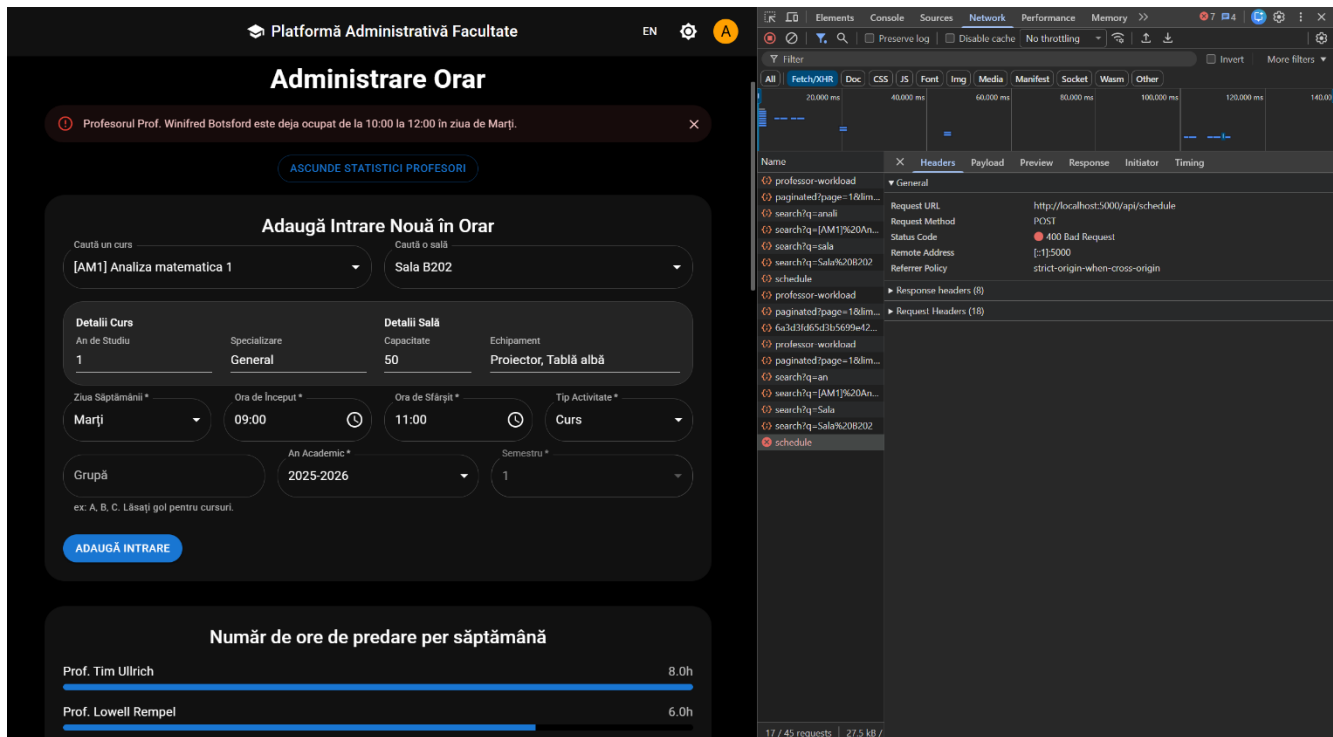


Figura 3.1 – Interceptarea și respingerea unei alocări conflictuale de către algoritm.

3.1.1. Refactorizare arhitecturală și principiul DRY

În faza inițială de dezvoltare a aplicației, logica matematică din spatele conversiilor temporale (exemplu: `parseTime` și `checkTimeOverlap`) a fost strâns legată de anumite module din partea de server. Totuși, o dată ce sistemul a fost scalat să includă și programări la secretariat, să genereze dinamic intervale de timp și să execute servicii CRON în fundal, această abordare a încălcat principiul din ingineria software DRY (Don't Repeat Yourself) [10]. Logica redundantă crește riscul apariției unui comportament inconsistent, și complică eventualele lucrări de mentenanță viitoare.

Pentru a impune separarea responsabilităților (SOC), toată logica legată de calcule temporale a fost decuplată din controlere și centralizată într-un serviciu dedicat de utilități (`timeUtils.js`).

În plus, algoritmul principal `parseTime` a fost îmbunătățit pentru a avea toleranță la posibile erori. Funcția îmbunătățită evaluează dinamic lungimea șirului de timp, și, în același timp, evaluează tipul de date primit (exemplu: funcția acceptă ca și parametri de intrare numere: `typeof ... === 'number'`, precum 9, 10; și, de asemenea, acceptă și șiruri de caractere: „9:15”, „14:30”, „17”, și returnează numărul de minute trecute de la începutul zilei, reprezentat de parametrul de intrare):

```
// Centralised Temporal Processing Utility
const parseTime = (timeInput) => {
  // Returns 0 for null/undefined type of data
  if (timeInput === undefined || timeInput === null) return 0;

  // Returns number * 60 for a number input
  if (typeof timeInput === 'number') {
    return timeInput * 60;
  }

  const timeStr = String(timeInput);

  // Dynamically destructure array to handle HH:mm format
  // Returns hour * 60 + minutes
  if (timeStr.includes(':')) {
    const [hours, minutes] = timeStr.split(':').map(Number);
    return (hours || 0) * 60 + (minutes || 0);
  }

  // Fallback, if input is not number, or in HH:mm format (like „9” string)
  const hours = parseInt(timeStr, 10);
  return isNaN(hours) ? 0 : hours * 60;
};
```

Centralizând logica îmbunătățită, oricare controler din server poate importa această funcție, garantând astfel că toate evaluările temporale făcute de această platformă sunt identice din punct de vedere matematic și pot analiza un spectru larg de parametri de intrare.

3.2. Inițializare automatizată a mediului și anonimizarea datelor

3.2.1. Paradigma inițializării

Generarea și lansarea unui sistem universitar complet necesită o stare inițială extrem de structurată a bazei de date. Astfel, este foarte ineficient și predispus la erori umane inițializarea manuală a unei baze de date (introducere manuală a datelor pentru a popula sute de utilizatori, săli și cursuri). Pentru a automatiza pipeline-ul de implementare, un serviciu robust de inițializare (`seeder.js`) a fost proiectat. Acest script are două funcționalități principale: inițializarea mediului de producție cu

credențiale administrative esențiale și generarea de date realiste și distribuite statistic pentru posibilitatea testării exhaustive a platformei.

Figura 3.2 expune fluxul de ieșire al scriptului de seeding, executat direct în interiorul containerului Docker aferent backend-ului. Log-urile confirmă vizual secvența de operațiuni a algoritmului: conectarea la baza de date, ștergerea asincronă a documentelor preexistente pentru a evita coliziunile și popularea iterativă a colecțiilor cu date generate dinamic. De asemenea, log-ul evidențiază instanțierea cursurilor cu profesor anonimizati, confirmând respectarea constrângerilor de confidențialitate și funcționarea logicii de afaceri implementate la nivel de server.

```
PS C:\Users\rober\Documents\MERN-Stack-University-Administration-App> docker-compose exec backend npm run seed

> faculty-admin-platform-backend@1.0.0 seed
> node seed/seeder.js

[dotenv@17.0.1] injecting env (0) from .env - [tip] encrypt with dotenvx: https://dotenvx.com
MongoDB Connected for seeding...
Clearing existing data...
Admin created
19 rooms created
30 teachers created
10 external representatives created
27 courses created with anonymized professors
160 students created (Years 1-4)
Generating schedule entries...
41 schedule entries generated
Seeding completed successfully!
```

Figura 3.2 – Log-ul modului de inițializare automatizată a datelor (seeding)

3.2.2. Anonimizarea dinamică a datelor și respectarea confidențialității

Un impas major întâlnit în faza de dezvoltare a fost importul de date academice reale (exemplu: cursuri academice deja existente și profesorii care predau aceste cursuri) fără a încălca standardele de confidențialitate a datelor sau a utiliza informații despre persoane reale într-un mediu de testare.

Pentru a rezolva acest impas, scriptul folosește librăria `@faker-js/faker` [11] pentru a realiza anonimizarea dinamică a datelor. Sistemul citește structura reală a curriculumului dintr-un fișier local (`courses.json`), dar și elimină numele real al profesorilor (Cu toate acestea, pentru a păstra confidențialitate absolută, nu este salvat niciun nume al unei persoane reale, întrucât, în fișier, toate numele au fost înlocuite cu “Anonim”. Totuși, funcționalitatea scriptului rămâne aceeași, pentru cazuri în care sunt importate fișiere care conțin date confidențiale). Acesta menține o tabelă hash (`professorMap`) care asignează o identitate fictivă, nou generată, de profesor. Acest procedeu asigură că orarul generat rămâne matematic consistent (unul sau mai mulți profesori, pentru curs, seminar și laborator sunt asignați la cursurile lor respective), dar toate datele cu caracter personal sunt complet șterse:

```
// Dynamic Anonymization and Relational Mapping Logic
const originalCourses = JSON.parse(fs.readFileSync(path.join(__dirname,
  'courses.json'), 'utf-8'));
```

```

const professorMap = {};
const getAnonymizedTeacher = (originalName) => {
  if (!originalName) return null;
  // If the real professor hasn't been mapped yet, assign a generated identity
  if (!professorMap[originalName]) {
    professorMap[originalName] =
    faker.helpers.arrayElement(createdTeachers);
  }
  // Return the anonymized username to be stored in the database
  return professorMap[originalName].username;
};

```

3.2.3. Logică de afaceri condiționată și integritate referențială

Secvența de inițiere impune cu strictețe logica de afaceri academică. De exemplu, bucla de generare a studenților aplică logică condițională pentru a determina specializarea academică: studenților de anul unu sau doi le este atribuită specializarea `General`, iar studenților de anul trei și patru sunt distribuiți dinamic între diferitele specializări valabile (ELA, MON, TST).

Mai mult, scriptul păstrează integritate referențială absolută asupra tuturor colecțiilor din baza de date. Când generează intrări aleatoare `ScheduleEntry`, algoritmul nu doar inserează șiruri izolate, ci acesta compară șirurile generate anterior (din memorie) pentru a extrage și încorpora referințele absolute `_id` ale `Mongoose` ale documentelor `Room` și `Course`.

3.2.4. Mecanisme asincrone de ștergere a datelor

Pentru a facilita iterația rapidă în timpul dezvoltării, scriptul include un indicator de distrugere (-d). Spre deosebire de procesul secvențial de inserție a datelor, mecanismul de ștergere optimizează timpul de execuție utilizând modelul de concurență `Promise.all` al `Node.js`:

```

// Concurrent Database Teardown
if (process.argv[2] === '-d') {
  connectDB().then(() => {
    console.log('Destroying data...');
    Promise.all([
      User.deleteMany(),
      Room.deleteMany(),
      Course.deleteMany(),
      RoomReservation.deleteMany(),
      ScheduleEntry.deleteMany(),
      Appointment.deleteMany()
    ]).then(() => {
      console.log('Data destroyed.');
```

```

      process.exit();
    });
  });
}

```

Folosind toate operațiile deleteMany simultan în motorul bazei de date, bucla de evenimente Node.js rămâne deblocată, obținând astfel o resetare performantă și instantanee a bazei de date.

3.3. Modulul de rezervări al sălilor și optimizarea interogărilor temporale

Cu toate că algoritmul de detecție a suprapunerilor pe 4 niveluri gestionează partea statică de intrări în baza de date (evenimente recurente/săptămânale, cum ar fi cursuri/laboratoare), platforma are nevoie și de un mecanism dinamic pentru rezervări spontane de săli (pentru pregătiri înainte de examen, evenimente externe, cum ar fi workshop-uri). Acest mecanism introduce un nou nivel de complexitate, care presupune validarea suprapunerii intervalelor de timp, în date exacte, non-recurente.

Un obstacol tehnic întâlnit în timpul implementării acestui nou modul a fost sincronizarea fusurilor orare dintre interfața utilizatorului și nivelul persistent al bazei de date. În mod implicit, MongoDB stochează obiectele de dată (Date) ca întregi de 64 de biți. Aceștia reprezintă numărul de milisecunde trecute de la punctul zero (timpul Unix – 1 ianuarie 1970, ora 00:00:00) în ora universală coordonată (UTC). Prin urmare, executând interogări cu potrivire exactă a datelor (exemplu: { date: requestedDate }) a rezultat în detecții de suprapunere eronate/false. Deoarece partea de client injectează/adaugă abateri de câteva milisecunde în timpul analizei datelor, baza de date va detecta suprapuneri false, chiar dacă rezervările se află la ore diferite (exemplu: 16:00-18:00; 18:00-20:00). Tehnic, rezervările sunt făcute la ore diferite, dar erorile de câteva milisecunde le vor face să se suprapună.

Pentru a proiecta o soluție corectă, dar nu atât de strictă, interogarea cu potrivire exactă de mai sus a fost înlocuită cu interogarea pe intervale. Mai exact, după ce server-ul (controlerul, mai specific) primește payload-ul, acesta construiește dinamic o structură de execuție temporal, absolut pentru ziua respectivă:

```
// Establish absolute lower boundary (Start of Day)
const startOfDay = new Date(date);
startOfDay.setHours(0, 0, 0, 0);

// Establish absolute upper boundary (End of Day)
const endOfDay = new Date(date);
endOfDay.setHours(23, 59, 59, 999);

// Execute bounded range query against persistent storage
const existingReservations = await RoomReservation.find({
  room,
  date: {
    $gte: startOfDay,
    $lte: endOfDay
  },
  status: { $in: [,'pending', , 'approved'] }
});
```


Folosind operatorii relaționali `>=` (mai mare decât sau egal cu) și `<=` (mai mic decât sau egal cu) [5], interogarea va găsi toate rezervările făcute în acea zi calendaristică. O dată ce acest set de date este primit și poate fi utilizat, server-ul poate aplica funcția `checkOverlap` pentru a preveni suprapunerile rezervărilor. Acest artificiu arhitectural izolează cu succes logica folosită pentru rezervări și detaliată mai sus de posibilele erori cauzate de schimbările fusului orar.

3.4. Generarea dinamică de intervale orare pentru rezervări la secretariat

O cerință de bază a unei platforme administrative este de a eficientiza cât mai mult posibil interacțiunea student-secretariat. Spre deosebire de cursurile academice obișnuite, care au durate de timp fixe, rezervările la secretariat necesită un mecanism de rezervări detaliat, la cerere. Pentru a oferi acest sistem, fără a suprapopula baza de date și fără a prezenta intervale orare indisponibile utilizatorului, un algoritm generator de intervale orare dinamice a fost construit în modulul de rezervări la secretariat (`appointmentController.js`).

3.4.1. Arhitectură bazată pe configurare și generator dinamic de intervale orare

Limitarea adusă de constante

În prima variantă a algoritmului de generare de programări administrative, parametrii operaționali, cum ar fi ora de deschidere a secretariatului, ora de închidere și durata unei programări, au fost incluse direct în cod ca și valori constante (exemplu: programul secretariatului era de la 09:00 la 17:00, cu intervale de 15 minute). Chiar dacă, în acest caz specific, această metodă este funcțională, această abordare cu parametri incluși în cod încalcă principiile scalabilității și modularității software. Dacă aplicația ar fi fost lansată pentru o universitate diferită, sau programul secretariatului ar fi diferit în timpul vacanței de vară, codul sursă ar necesita modificări și recompilări manuale.

Implementare de abstractizare la nivel de mediu

Pentru a soluționa această nouă problemă, algoritmul de generare de intervale orare a fost refactorizat pentru a utiliza o arhitectură bazată pe configurare. Respectând metodologia „Twelve-Factor App” [12], toate constrângerile operaționale au fost înlăturate din codul sursă Node.js și injectat la momentul execuției prin intermediul variabilelor de mediu ale sistemului (`.env`).

Această abordare le permite administratorilor de sistem să modifice comportamentul întregii platforme doar prin simpla modificare a acestui fișier, fără a fi nevoiți să atingă codul aplicației:

```
// Environment variables injected at runtime
// process.env.SECRETARITAT_START_HOUR = 9
// process.env.SECRETARITAT_END_HOUR = 17
// process.env.SECRETARITAT_APPOINTMENT_DURATION = 15

const allSlots = [];
const startMinutes = parseTime(process.env.SECRETARITAT_START_HOUR || „09:00”);
const endMinutes = parseTime(process.env.SECRETARITAT_END_HOUR || „17:00”);
const duration = parseInt(process.env.SECRETARITAT_APPOINTMENT_DURATION || 15, 10);
```

```

for (let current = startMinutes; current < endMinutes; current += duration) {
  // Calculate the mathematical start time
  const h = Math.floor(current / 60);
  const m = current % 60;
  const slotStart = `${h.toString().padStart(2, '0')}:${m.toString().padStart(2, '0')}`;

  // Calculate the mathematical end time based on the dynamic duration
  const next = current + duration;
  const nextH = Math.floor(next / 60);
  const nextM = next % 60;
  const slotEnd = `${nextH.toString().padStart(2, '0')}:${nextM.toString().padStart(2, '0')}`;

  allSlots.push({ startTime: slotStart, endTime: slotEnd });
}

```

Execuție algoritmică și toleranță la erori

În timpul execuției, controlerul interoghează obiectul `process.env`. Pentru a păstra stabilitatea sistemului și în eventuala situație în care nu există fișier de configurare, algoritmul este dotat cu operatori logici de rezervă (`||`), asigurând că sistemul va trece la orele de funcționare standard în loc să nu mai funcționeze.

Funcția `parseTime` transformă variabilele de configurare din numere naturale, sau șiruri numerice în numărul de minute trecute de la începerea zilei. Apoi, bucla principală `for` iterează peste toate intervalele orare disponibile. Foarte important este că, operatorul de incrementare a buclei este dependent de variabila dinamică `SECRETARIAT_APPOINTMENT_DURATION`.

În interiorul buclei, modulul matematic (%) și operatorul de diviziune reconstruiesc valoarea primită, în minute, înapoi în numere întregi, reprezentând ora și minutul. În final, metoda nativă JavaScript `String.prototype.padStart()` este aplicată pentru a garanta că numerele întregi de o singură cifră, obținute mai sus, sunt formatate corect cu un zero la început (exemplu: convertind 9:00 la standardul 09:00). Această generare dinamică are ca rezultat un șir simetric de intervale orare, complet dictat de configurația serverului extern.

Eficiența algoritmului de calcul temporal este ilustrată în Figura 3.3, care reprezintă interacțiunea în timp real dintre client și server în cadrul modulului de programări la secretariat. În panoul instrumentelor de dezvoltare (dreapta), se observă răspunsul endpoint-ului `available-slots` în urma unei cereri HTTP `GET`. Controller-ul Express.js nu interoghează un tabel static, ci calculează dinamic intervalele disponibile de câte 15 minute, excluzând asincron programările deja confirmate în baza de date. Payload-ul este returnat sub forma unui șir JSON strict structurat. Simultan, pe partea de client (stânga), componenta React consumă acest răspuns și randează condiționat butoanele de selecție orară, asigurând astfel o sincronizare perfectă între disponibilitatea reală a secretariatului și interfața utilizatorului.

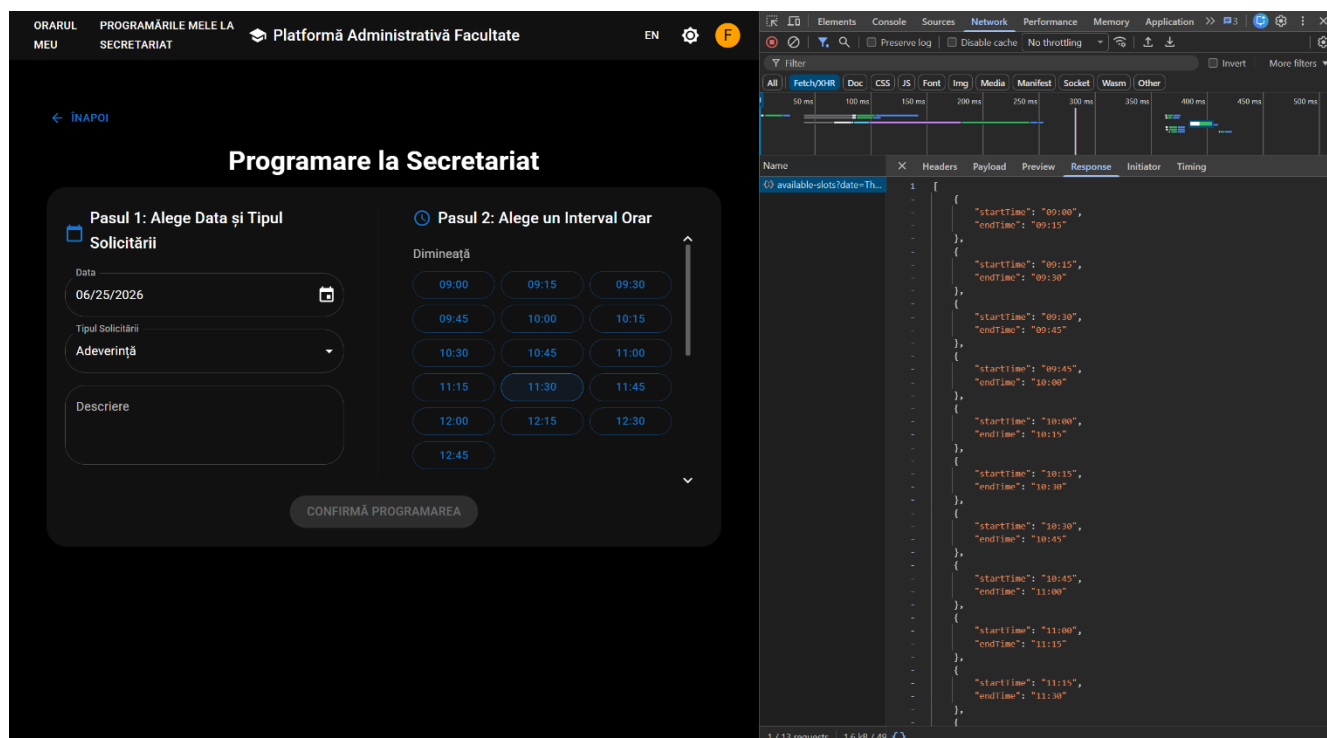


Figura 3.3 – Generarea dinamică a intervalelor orare și structura răspunsului JSON

3.4.2. Filtrarea proactivă a suprapunerilor

Generarea tabloului bidimensional este doar prima parte a acestui algoritm. Înainte de a returna datele payload-ul către utilizator, sistemul filtrează strict șirul `allSlots` generat de prima parte a algoritmului, folosind datele din nivelul persistent al bazei de date.

Server-ul execută o interogare specifică care va returna doar rezervările care sunt în așteptare, sau confirmate, pentru ziua cerută, omițând special rezervările anulate sau expirate. Intervalele neprelucrate, abia generate, sunt apoi procesate succesiv față de înregistrările din baza de date:

```
const availableSlots = allSlots.filter(slot => {
  for (let existingAppt of appointmentsForDay) {
    if (checkTimeOverlap(existingAppt.startTime, existingAppt.endTime,
      slot.startTime, slot.endTime)) {
      return false; // Slot is intersecting and removed from the available
      array
    }
  }
  return true; // Slot is validated and appended to the client payload
});
```

Această metodă de filtrare proactivă garantează faptul că utilizatorul va primi o varietate de opțiuni validate anterior de server. Rezolvând conflictele temporale doar pe partea de server înainte de a randa interfața utilizatorului, elimină posibilitatea ca un interval orar să fie rezervat de doi utilizatori în același timp. De asemenea, în acest mod este redus semnificativ numărul de cereri pe care utilizatorul îl va face către server, doar pentru a găsi un interval de timp valabil.

3.5. Automatizare în fundal și întreținerea bazei de date

3.5.1. Necesitatea unui sistem autonom de mentenanță

În lucrul cu o platformă administrativă scalabilă, este inefficientă bazarea exclusivă pe solicitări HTTP ale utilizatorului pentru actualizarea stărilor bazei de date. Unele tranziții de stări, precum actualizând o rezervare ca și completată odată ce ora de final a acesteia a trecut, sau actualizând o rezervare în așteptare, care nu a fost aprobată, ca și expirată, trebuie să aibă loc autonom bazat pe trecerea timpului. Pentru a rezolva această problemă, a fost folosită librăria `node-cron` [6] pentru a implementa, pe server, un planificator de activități în fundal. Acesta desparte operațiile de mentenanță de fluxul principal de comunicare între client și server.

3.5.2. Schimbările de stare bazate pe timp și execuția în fiecare oră

Logica principală a automatizării se află în modulul `cronJobs.js`. Acesta inițializează un proces de fundal (daemon) CRON configurat cu o expresie de execuție în fiecare oră (`0 * * * *`), spunându-i buclei de evenimente Node.js să ruleze automat scriptul de mentenanță la începutul fiecărei ore.

Acest script vizează două colecții principale: `Appointments` (rezervări secretariat) și `RoomReservation` (rezervări săli). Pentru ambele entități, sistemul evaluează două tranziții distincte de stări:

1. Expirare: Dacă o solicitare rămâne în starea de `pending`, iar timpul actual trece de timpul de început al programării, starea va fi schimbată în `expired`.
2. Finalizare: Dacă o solicitare este `confirmed/approved`, iar timpul actual trece de timpul de sfârșit al programării, starea va fi schimbată în `completed`.

3.5.3. Pipeline de agregare pentru evaluare temporală

Deoarece schemele din baza de date separă data din calendar (care este stocată ca și marcaj temporal UTC) de orele lucrătoare ale zilei (stocate ca șiruri de tipul `HH:mm/oo:mm`), comparația standard MongoDB pentru date nu este suficient de precisă pentru comparații la nivel de oră.

Pentru a putea obține precizie la nivel de minut fără a încărca o mulțime de documente în memoria mediului Node.js, scriptul de automatizare folosește pipeline-uri de agregare avansate, cu ajutorul operației `updateMany`. Această operație construiește dinamic un obiect ce conține atât data, cât și ora exactă pe partea de server, în baza de date:

```
// Example of the Aggregation Pipeline used in the Cron Job
await Appointment.updateMany(
  { status: ,pending', date: { $lte: now } },
  [{
    $set: {
      status: {
        $cond: {
          if: {
            $lte: [
```

```

        // Dynamically concatenate Date + „T” + startTime +
        „:00Z”

        { $dateFromString: { dateString: { $concat: [
            { $dateToString: { format: „%Y-%m-%d”, date: „$date”
            „T”, „$startTime”, „:00Z”
            ]}}},
            now
        ]
        },
        then: „expired”,
        else: „$status”
    }
}
}
}]]
);

```

În timpul execuției, operatorul `$concat` concatenează șirul formatat (ce conține data) cu șirul `startTime`, care este apoi analizat și convertit într-un obiect temporal (comparabil cu ce avem mai sus) prin `$dateFromString`. Operatorul `$cond` (condițional) compară noul obiect temporal creat cu variabila `now`. Dacă evaluarea este adevărată, starea este actualizată deodată.

Această structură păstrează toate operațiile complexe în motorul bazei de date (MongoDB) prin intermediul operațiilor descrise mai sus, și se asigură că platforma rămâne la fel de performantă, chiar și când va fi executat codul de curățare, necontând volumul de date din interiorul bazei de date.

3.6. Hashing criptografic și autentificare fără stări

3.6.1. Apărare împotriva vulnerabilităților criptografice

O cerință fundamentală a oricărei platforme administrative este necesitatea securității datelor de autentificare ale utilizatorului. Stocarea parolelor folosind text sau folosind algoritmi învechiți de hashing (cum ar fi MD5 sau SHA-1) expune baza de date către atacuri de tip dictionary sau atacuri de tip tabelă curcubeu în cazul unei breșe de securitate. Pentru a întări securitatea datelor, platforma utilizează librăria `bcryptjs` pentru a implementa salting criptografic sau extinderea cheii.

În loc să gestioneze logica de criptare în interiorul controlerului de autentificare, algoritmul criptografic este integrat la nivelul de date folosind un interceptor `pre('save')` al Mongoose. Acest procedeu asigură ca parola nu este niciodată introdusă în baza de date fără a fi mai întâi criptată, fără a conta unde își are originea evenimentul de creare al parolei:

```

// Mongoose Lifecycle Hook for Cryptographic Hashing
userSchema.pre('save', async function(next) {

    // Bypass encryption if the password field has not been modified
    if (!this.isModified('password')) {

```

```

    return next();
  }
  try {
    // Generate a cryptographic salt with a work factor of 10
    const salt = await bcrypt.genSalt(10);
    this.password = await bcrypt.hash(this.password, salt);
    next();
  } catch (error) {
    next(error);
  }
});

```

O componentă critică a apărării împotriva vulnerabilităților (Figura 3.4) o reprezintă validarea strictă a identității la nivelul fiecărui endpoint protejat de pe server. Deoarece arhitectura frontend (React) blochează proactiv utilizatorii neautentificați și îi redirecționează, testarea siguranței backend-ului a impus ocolirea interfeței grafice și atacarea directă a API-ului folosind platforma Postman. Captura ilustrează comportamentul sistemului defensiv atunci când o cerere HTTP GET este trimisă către o rută protejată (/api/room-reservations), având un header x-auth-token manipulat intenționat. Middleware-ul de securitate din Express.js interceptează pachetul de date înainte de a atinge logica de control, nu poate decoda semnătura JWT și returnează instantaneu codul HTTP 401 (Unauthorized), alături de mesajul descriptiv “Token is not valid”. Această barieră arhitecturală, de tip apărare în profunzime, garantează imposibilitatea extragerii sau modificării datelor din exteriorul aplicației, chiar și în eventualitatea compromiterii nivelului de prezentare vizuală.

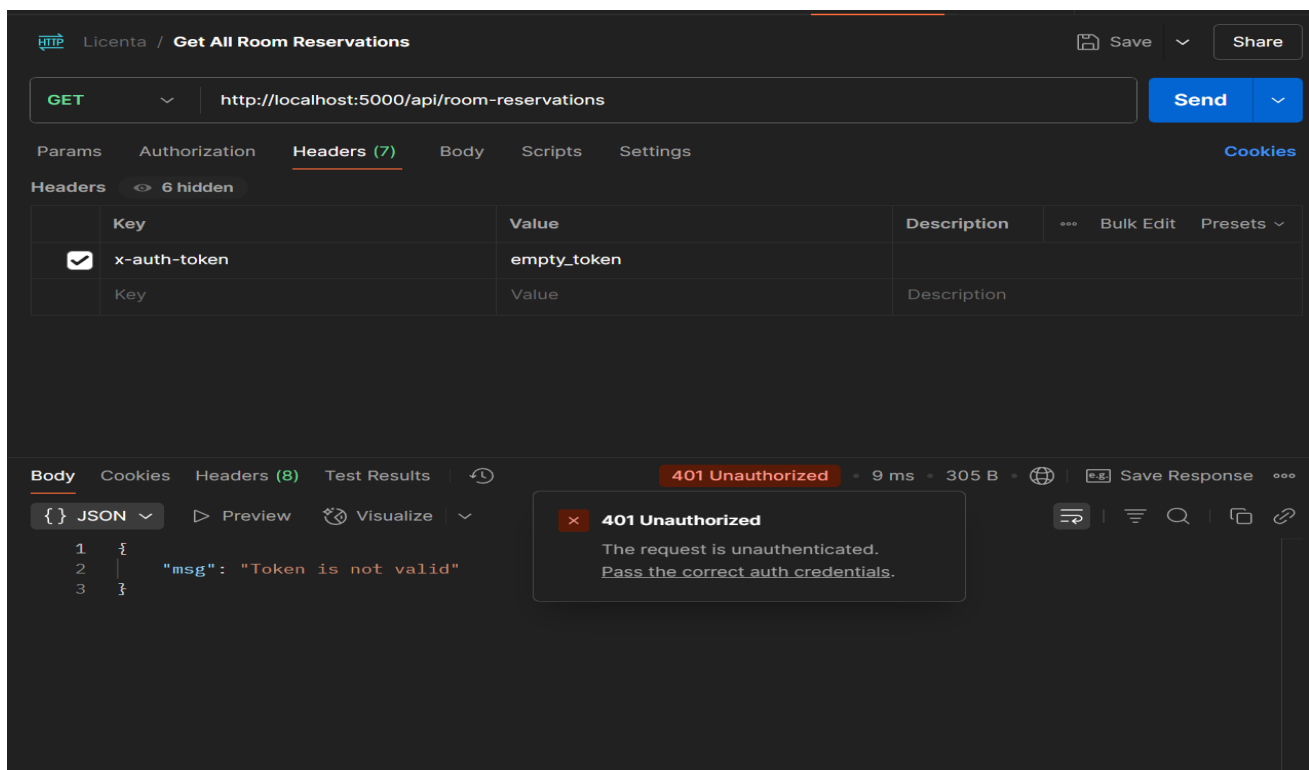


Figura 3.4 – Testarea securității endpoint-urilor utilizând clientul API Postman

3.6.2. Factorii de lucru algoritmici și fenomenul de salting

Execuția comenzii `bcryptjs.genSalt(10)` aplică un factor de lucru computațional (2^{10} iterații) procedului de hashing. Acest proces încetinește generarea criptografică. În timp ce o întârziere de câteva milisecunde este imperceptibilă pentru un utilizator care se autentifică normal, aceasta va degrada semnificativ performanța unui atac de tip brute-force, făcând astfel decriptarea offline matematic imposibil de realizat. În plus, salt-ul generat aleatoriu asigură că două parole identice ale unor utilizatori distincți vor avea semnături hash complet diferite.

La nivelul bazei de date (Figura 3.5), se confirmă vizual ireversibilitatea algoritmului de securitate implementat. Parola utilizatorului nu este stocată în forma introdusă de acesta, ci sub forma unui hash criptografic. Prefixul `$2b$10$`, vizibil în câmpul alocat parolei, indică utilizarea algoritmului `bcrypt` cu un factor de cost de 10. Această abordare atașează automat un salt aleatoriu înainte de dispersie, eliminând riscul atacurilor cu tabele precalculate. De asemenea, documentul BSON ilustrează definirea explicită a atributului `role`, ca fiind `teacher`. Aceasta este o informație esențială, care va fi inclusă în payload-ul token-ului JWT pentru aplicarea politicilor de control al accesului (RBAC).

```
_id: ObjectId('6a3d4a47b968e84a6ef64a75')
username: "Asist. Michael Walsh"
email: "michael_walsh@gmail.com"
password: "$2b$10$vixTbOmb4gMCWx4uv1WX3eUayymzSpRqWwTE2NDSZ4xAUrzRsRnG"
role: "teacher"
createdAt: 2021-12-06T13:02:39.901+00:00
__v: 0
```

Figura 3.5 – Stocarea securizată a credențialelor utilizând algoritmul `bcrypt`

3.7. Validare temporală între colecții

3.7.1. Provocarea adusă de planificare cu contexte multiple

Într-un sistem cu rezervări obișnuite, verificarea valabilității înseamnă, de obicei, interogarea unei singure colecții (exemplu: verificarea unei noi rezervări față de rezervările deja existente). Totuși, mediul universitar operează după un model cu dublu context: valabilitatea unei săli este dictată și de o solicitare singulară administrativă (`RoomReservation`), și de un orar academic fix și recurent (`ScheduleEntry`).

Pentru a evita conflictele fizice legate de săli, algoritmul de rezervări trebuie să creeze o matrice unificată de detecție a coliziunilor pe mai multe scheme separate ale bazei de date.

3.7.2. Realizarea algoritmică a matricei

În momentul în care un profesor trimite o cerere de rezervare, controlerul din server (`roomReservationController.js`) izolează, mai întâi, limitele temporale, normalizând cererea la standardul UTC (folosind `startOfDay` și `endOfDay`).

Apoi, algoritmul execută prima etapă de validare, interogând colecția `RoomReservation` pentru detecție de suprapuneri. Dacă această etapă este validată, sistemul trece la a doua etapă, cea între colecții.

Astfel, determină matematic ziua săptămânii curentă (exemplu: 'Monday') din obiectul `Date` localizat al utilizatorului și interoghează colecția `ScheduleEntry` pentru a extrage orarul academic fix pentru acea specifică sală:

```
// Phase 2: Cross-Collection Validation against the Academic Timetable
const days = [ ,Sunday' , ,Monday' , ,Tuesday' , ,Wednesday' , ,Thursday' , ,Friday' ,
,Saturday' ];
const dayOfWeek = days[startOfDay.getDay()];

const academicScheduleEntries = await ScheduleEntry.find({
  room,
  dayOfWeek,
});

for (let entry of academicScheduleEntries) {
  if (checkTimeOverlap(entry.startTime, entry.endTime, startTime, endTime))
  {
    const course = await Course.findById(entry.course);
    return res.status(400).json({
      msg: ,RESERVATION_BLOCKED_BY',
      details: {
        roomName: existingRoom.name,
        courseName: course ? course.name : ,Unknown Course',
        startTime: entry.startTime,
        endTime: entry.endTime,
        dayOfWeek: dayOfWeek
      }
    });
  }
}
```

Trecând ambele seturi de date distincte prin funcția matematică centralizată `checkTimeOverlap`, serverul se comportă ca un firewall extrem de robust, pe mai multe niveluri. Dacă este detectată o suprapunere în una din cele două colecții, execuția codului este întreruptă instant și este construit un payload dinamic și foarte specific (`RESERVATION_BLOCKED_BY`), care informează clientul exact care curs sau care utilizator cauzează conflictul temporal.

Capitolul 4. Interfața grafică și experiența utilizatorului

4.1. Arhitectura componentelor și administrarea stărilor

Frontend-ul aplicației este creat ca și o aplicație pe o singură pagină (SPA) folosind React.js. Această arhitectură se asigură ca browserul nu este nevoit să reîncarce toată pagina când utilizatorul navighează prin diferite secțiuni, rezultând astfel o experiență rapidă și fluidă pentru utilizator.

Interfața utilizatorului este construită utilizând componente modale, reutilizabile. Pentru a administra starea globală a aplicației (de exemplu: ce utilizator este autentificat, limba selectată: engleză/română, și tema vizuală: mod luminos/întunecat), a fost folosit API-ul Context, încorporat în React.

Codul de dedesubt arată funcția `ColorModeProvider`, creată special pentru a gestiona comutarea între modul luminos/întunecat. De asemenea, este încorporat și `localStorage`. Astfel, preferința vizuală a utilizatorului este fizic salvată în memoria browserului, și automat utilizată în următoarele sesiuni, având astfel o experiență persistentă a utilizatorului.

```
// ThemeContext.jsx
import React, { createContext, useState, useMemo, useContext } from 'react';

// Create a global context with default values
export const ColorModeContext = createContext({
  toggleColorMode: () => {},
  mode: 'light',
});

export const useColorMode = () => useContext(ColorModeContext);

export const ColorModeProvider = ({ children }) => {
  // Initialize state by checking local browser memory, defaulting to 'light'
  const [mode, setMode] = useState(localStorage.getItem('themeMode') || 'light');

  // useMemo optimizes performance by caching the function
  const colorMode = useMemo(
    () => ({
      toggleColorMode: () => {
        setMode((prevMode) => {
          const newMode = prevMode === 'light' ? 'dark' : 'light';
          localStorage.setItem('themeMode', newMode); // Save user preference
          return newMode;
        });
      },
      mode,
    }),
    [mode],
  );
};
```

```

    return (
      <ColorModeContext.Provider value={colorMode}>
        {children}
      </ColorModeContext.Provider>
    );
  };
};

```

Îmbrăcând aplicația principală în acest ThemeProvider, orice componentă (ca de exemplu: bara de navigație, meniul de setări) pot accesa și modifica tema vizuală imediat, fără a trece informație prin mai multe niveluri.

4.2. Material UI și design reactiv

Pentru a avea un model vizual consistent și profesional, librăria de componente Material UI (MUI) a fost integrată. Aceasta înlocuiește scrierea de cod CSS pentru fiecare buton și tabel, aducând componente preconstruite și personalizabile.

Un principiu de bază al acestei platforme este menținerea unei experiențe SPA, în timp ce utilizatorul poate naviga prin liste/tabele administrative complexe. Mai mult, interfața necesită suport pentru internaționalizare (posibilitatea de a schimba limba între engleză și română).

Componenta personalizată BackButton este un exemplu perfect pentru cele discutate mai sus:

```

// BackButton.jsx
import React from 'react';
import { useNavigate } from 'react-router-dom';
import { Button } from '@mui/material';
import ArrowBackIcon from '@mui/icons-material/ArrowBack';
import { useTranslation } from 'react-i18next';

const BackButton = ({ to }) => {
  // Hook for SPA routing (manipulating browser history)
  const navigate = useNavigate();
  // Hook for dynamic language translation
  const { t } = useTranslation();

  const handleClick = () => {
    if (to) {
      navigate(to); // Navigate to a specific path without refreshing the
page
    } else {
      navigate(-1); // Automatically go back one step in browser history
    }
  };

  return (
    <Button
      variant="text"
      startIcon={<ArrowBackIcon />}

```

```

    onClick={handleClick}
    // The ,sx' prop is MUI's system for rapid, inline responsive styling
    sx={{ mb: 2, alignSelf: ,flex-start' }}
  >
    {/* Dynamically loads ,Back' or ,Înapoi' based on active language */}
    {t(,back_button')}
  </Button>
);
};

export default BackButton;

```

În această arhitectură:

- Rutare: Interceptorul `useNavigate`, preluat din React Router, asigură că navigatul înapoi este procesat imediat de JavaScript, în loc de a fi solicitată o nouă pagină HTML de la server.
- Localizare: Interceptorul `useTranslation` (`t`) se conectează direct la starea globală de limbă, actualizând imediat textul butonului când utilizatorul schimbă limba.
- Stilizare: Proprietatea `sx` este o caracteristică specifică librăriei Material UI, care permite aplicarea de stilizare dinamică (de exemplu: `mb: 2` pentru marginea de jos) direct în logica componentei, fără a mai fi nevoie de fișiere separate CSS.

Figura 4.1 ilustrează comportamentul adaptiv al platformei atunci când este accesată de pe un dispozitiv mobil. Prin utilizarea sistemului de grile flexibile și a interogărilor media încorporate în stiva Material-UI, elementele interfeței se recalibrează dinamic. Se observă coomutarea automată a barei de navigare într-un format compact, care ascunde textul neesențial pentru a prioritiza acțiunile de bază, și redimensionarea fluidă a componentelor complexe de vizualizare a datelor (graficul cu bare). De asemenea, panourile informaționale abandonează așezarea orizontală în favoarea unei alinieri verticale, garantând o experiență de utilizare coerentă și accesibilă, independent de constrângerile fizice ale ecranului utilizatorului.

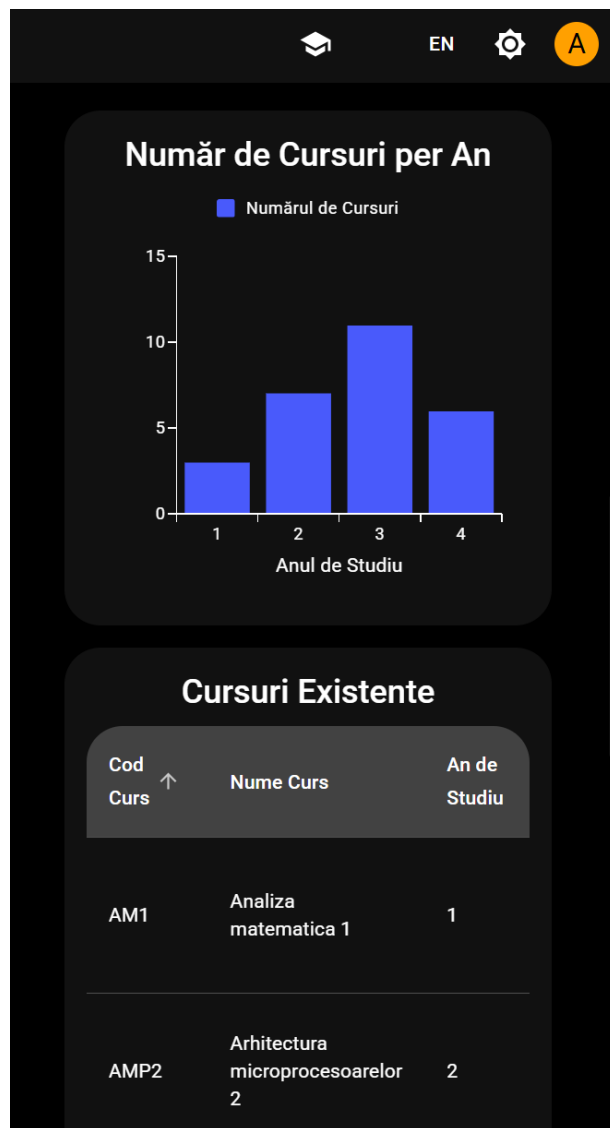


Figura 4.1 – Adaptabilitatea interfeței grafice implementată pe un viewport mobil

4.3. Optimizarea traficului de date și paginare gestionată de server

4.3.1. Problemele de randare a DOM (Document Object Model)

Pentru o aplicație pe o singură pagină (SPA), datele procurate de server sunt menținute în memoria locală a utilizatorului și generate în DOM-ul browserului. Pentru operații simple, această arhitectură asigură o experiență fără probleme a utilizatorului. În schimb, va introduce probleme mari de performanță încercând să gestioneze seturi de date administrative. Dacă sistemul încearcă să încarce mii de intrări din orar, sau date despre utilizatori, totul simultan, payload-ul imens (sub formă de JSON) va crește timpul de răspuns al aplicației, va crește consumul de memorie al utilizatorului, și va bloca interfața grafică a utilizatorului.

Pentru a rezolva aceste constrângeri și a menține viteza de răspuns a componentelor Material UI, procesarea de date trebuie să fie trimisă de la browserul utilizatorului către baza de date/server.

O metodă complementară paginării pentru prevenirea blocajelor la nivelul DOM-ului este filtrarea contextuală strictă, care limitează drastic volumul de informație trimis către client.

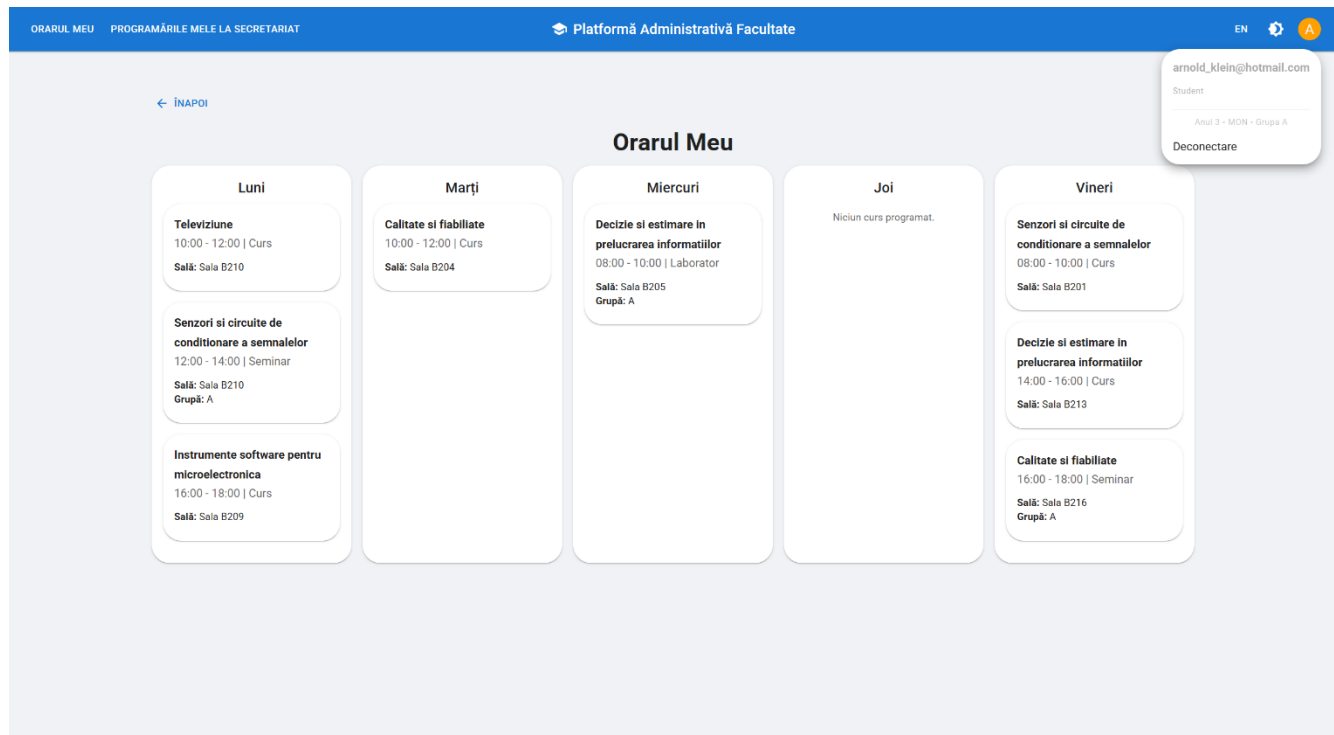


Figura 4.2 – Evitarea supraîncărcării DOM-ului prin randarea unui set de date redus, filtrat contextual

Figura 4.2 demonstrează vizual acest concept de optimizare. În loc să solicite și să randeze întregul orar al facultății, componenta React interoghează backend-ul folosind atributele unice ale utilizatorului (Anul 3, Specializare MON, Grupa A). Serverul returnează exclusiv intersecția de date relevantă, rezultând într-un payload JSON minimal. La nivelul browser-ului, DOM-ul trebuie să construiască și să gestioneze doar un număr redus de noduri (cardurile individuale ale cursurilor din acea săptămână). Această decuplare și filtrare menține performanța interfeței, demonstrând că o arhitectură eficientă transferă sarcina computațională de la procesorul clientului către motorul bazei de date.

4.3.2. Pipeline-uri de agregare ale bazei de date

O altă metodă prin care server-ul rezolvă problemele menționate mai sus este prin implementarea paginării gestionată de server, folosind pipeline-urile de agregare ale MongoDB. În loc să trimită o întreagă colecție de date către client, controller-ul folosește operatorul `$facet` pentru a gestiona mai multe pipeline-uri secundare [7].

Așa cum este implementat în `getPaginatedAppointments` (`appointmentController.js`) și în `getPaginatedSchedule` (`scheduleController.js`), pipeline-ul face joncțiuni relaționale folosind operatorii `$lookup` și `$unwind`. Pe urmă, calculează dinamic ordinea de sortare înainte de a realiza paginarea:

```
// Extract from appointmentController.js demonstrating the $facet pipeline
const results = await Appointment.aggregate([
  ...pipeline, // Previous $lookup and sorting stages
  {
    $facet: {
      data: [
        { $skip: (pageNum - 1) * limitNum },
        { $limit: limitNum },
        { $project: { statusOrder: 0 } } // Clean up temporary sorting
      ],
      pagination: [{ $count: 'totalItems' }]
    }
  }
]);
```

Această structură este foarte optimizată, deoarece pipeline-ul secundar de date utilizează operatorii \$limit și \$skip pentru a extrage exact numărul de blocuri de date din ce a cerut utilizatorul (în acest exemplu, exact 10 rânduri pentru pagina 2). Simultan, pipeline-ul secundar de paginare calculează numărul total de date care îndeplinesc sortarea/filtrarea activă. Astfel, este posibil ca server-ul să returneze o submulțime de date, împreună cu metadatele de paginare necesare clientului.

4.3.3. Integrarea în client și re-randarea dinamică a componentelor

Pe partea de client, componenta PaginatedTable folosește acest payload optimizat, descris în subcapitolul 4.3.2. Această componentă este gândită să funcționeze ca un receptor activ. Ea interceptează interacțiunile utilizatorului cu interfața grafică, precum apăsarea butonului “Pagina Următoare”, sau apăsarea unui titlu de tabel pentru a realiza sortarea după acel criteriu. După ce interceptează această interacțiune, trimite o nouă solicitare Axios către server.

Deoarece payload-ul primit este limitat la fereastra de vizualizare a utilizatorului (exemplu: 10 linii dintr-un tabel), DOM-ul virtual al React-ului poate reconstrui interfața tabelului aproape instantaneu. Această relație dintre motorul de agregare MongoDB și administratorul de stare React asigură că platforma poate menține o performanță de randare de ordinul $O(1)$ pe dispozitivul utilizatorului, chiar dacă baza de date conține o sută de mii, sau chiar un milion de intrări.

Această abordare de optimizare a traficului de rețea și de menținere a performanței de randare este vizibilă în figura 4.3.

Utilizatori Existenți						
Nume Utilizator ↑	Email	Rol	An de Studiu	Specializare	Grupă	Acțiuni
Abel_Walsh-Bode96	abel.walsh-bode41@gmail.com	Student	2	General	C	 
Al_Predovic96	al_predovic@hotmail.com	Student	4	ELA	B	 
Alicia_Ernser33	alicia_ernser@yahoo.com	Student	3	CTI	A	 
Alma.Kiehn50	alma.kiehn11@gmail.com	Student	3	MON	A	 
Ann_Balistreri	ann_balistreri11@yahoo.com	Student	1	General	B	 
Annie_Kreiger	annie_kreiger11@gmail.com	Student	1	General	C	 
Antonia_Ernser67	antonia.ernser@yahoo.com	Student	2	General	D	 
Asist. Francisco Ruecker	francisco_ruecker58@yahoo.com	Profesor	N/A	N/A	N/A	 
Asist. Jim Lehnner	jim_lehnner16@yahoo.com	Profesor	N/A	N/A	N/A	 
Asist. Kenny Mayer	kenny_mayer@yahoo.com	Profesor	N/A	N/A	N/A	 

< 1 2 3 4 5 _ 21 >

Figura 4.3 – Interfața componentei tabelare cu paginare gestionată de server

4.4. Gestionarea dinamică a erorilor și interceptarea i18n

4.4.1. Separarea logicii de bază a platformei și a limbii de prezentare

Într-o aplicație normală, omogenă, server-ul generează mesaje de eroare statice, ușor de citit de utilizator (exemplu: „Sala este rezervată”). Totuși, pentru o aplicație SPA accesibilă la nivel internațional, generarea de răspunsuri statică la nivel de server încalcă principiul separării responsabilităților (SoC).

Pentru a crea o arhitectură de internaționalizare (i18n) scalabilă, sistemul folosește un protocol semantic de gestionare al erorilor. În detaliu, când server-ul detectează un conflict/o eroare (exemplu: o suprapunere în orar), nu returnează un răspuns static (exemplu: „A fost detectată o suprapunere!”). În schimb, acesta oprește execuția programului și returnează un payload JSON care conține un cod de eroare standard HTTP 400 (Bad request), un identificator generic (`msg`), și un obiect încorporat (`details`) care conține următoarele constrângeri:

```
// Backend JSON Payload Example
return res.status(400).json({
  msg: 'RESERVATION_BLOCKED_BY',
  details: {
    roomName: existingRoom.name,
    courseName: course.name,
```

```

        startTime: entry.startTime,
        endTime: entry.endTime,
        dayOfWeek: dayOfWeek
    }
});

```

4.4.2. Interceptarea pe parte de client și maparea dinamică a cheilor

Pe partea de client, aplicația folosește blocuri `catch` în funcțiile existente pentru a intercepta răspunsurile HTTP detaliate la punctul 4.4.1. În loc să randeze mesajul intact primit de la server, clientul se comportă ca un intermediar de traducere.

Astfel, aplicația extrage identificatorul `msg`, îl convertește din litere mari în litere mici pentru a-l putea compara cu cheile JSON din fișierul `i18n.js` care conține toate traducerile în limba română și limba engleză, și stochează datele din obiectul `details` în administratorul de stare al React. Această structură asigură că maparea erorilor trimise de server este realizată cu succes.

```

// Frontend Interception Strategy
} catch (err) {
    const errorData = err.response?.data;

    if ([, 'RESERVATION_ROOM_RESERVED', 'RESERVATION_BLOCKED_BY'].includes(errorData?.msg)) {
        const errorKey = errorData.msg.toLowerCase();
        setModalError({ key: errorKey, type: 'error', details: errorData.details });
    }
}

```

4.4.3. Injecție dinamică a parametrilor folosind `react-i18next`

Ultima etapă a ciclului de gestionare a erorilor constă în randarea șirurilor localizate pentru utilizator. Platforma utilizează librăria `react-i18next` pentru a verifica fișierele utilizate pentru traducere [8]. Așadar, implementarea injecției dinamice a parametrilor reprezintă o realizare tehnică majoră a acestui capitol.

Șirurile de traducere sunt construite cu înlocuitori de interpolare (ex: “reservation_blocked_by”: „Room {{roomName}} is blocked by an academic class (, {{courseName}}’) from {{startTime}} to {{endTime}} on {{dayOfWeek}}.”). În momentul în care componenta `<Alert>` a Material UI randează eroarea, apelează funcția de traducere `t()` și propagă obiectul `details` obținut înainte în parametrul secundar de argumente:

```

// Dynamic UI Rendering
{modalError && modalError.key && (
    <Alert severity={modalError.type} || „error” sx={{ mb: 2 }}>
        {t(modalError.key, {
            ...modalError.details,

```



```

        //Nested dynamic translation for the day of the week
        dayOfWeek:modalError.details?.dayOfWeek
        t(`day_${modalError.details.dayOfWeek}`) : ,'
    }}
    </Alert>
  ) }
}

```

Această integrare permite clientului să construiască dinamic propoziții și fraze complexe și corect gramaticale în limba română și limba engleză. În plus, gestionează corect și traduceri imbricate (ex: preia termenul „Monday” de pe partea de server și îl traduce în termenul „Luni”, înainte de a-l injecta în șirul de caractere final). Această arhitectură aduce deci o integrare perfectă între server și client, oferind o experiență cursivă și neîntreruptă utilizatorului.

Figura 4.4 ilustrează convergența a două concepte arhitecturale majore detaliate în acest capitol: gestionarea stărilor de eroare complexe și internaționalizarea (i18n). În momentul în care utilizatorul încearcă să rezerve o sală într-un interval orar deja alocat, sistemul interceptează eroarea de suprapunere temporală. Modul de afișare la nivelul interfeței grafice confirmă implementarea cu succes a i18n: mesajul din componenta de alertă nu este un text static, ci un șablon extras din dicționarele bibliotecii `react-i18next` pentru limba activă (engleză). Parametrii specifici conflictului, numele sălii, materia, și intervalul orar, sunt injectați dinamic (prin interpolare de variabile) în acest șablon la momentul rulării. Astfel, aplicația SPA oferă un feedback localizat, contextual și precis, evitând expunerea utilizatorului la erori brute de server.

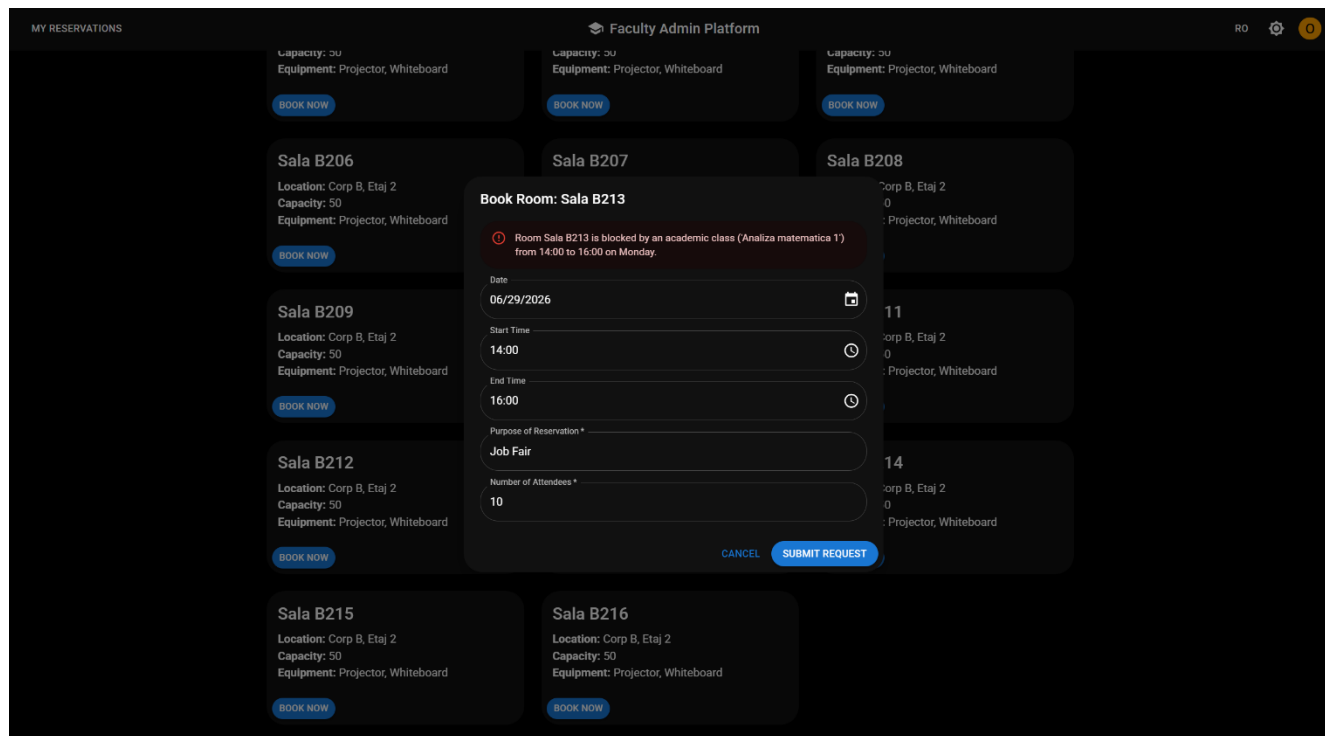


Figura 4.4 – Interceptarea erorilor de validare și injectarea dinamică a dicționarelor de traducere

În completarea mecanismelor de tratare a erorilor, aplicația gestionează cu aceeași rigoare stările de succes ale tranzacțiilor asincrone. Figura 4.5 ilustrează comportamentul interfeței în urma înregistrării valide a unei programări la secretariat. La recepționarea unui cod de status HTTP din clasa 2xx, componenta React actualizează starea globală pentru a declanșa o notificare vizuală de tip Alert. Acest feedback proactiv confirmă utilizatorului finalizarea operațiunii la nivelul bazei de date și anunță redirectionarea programatică (gestionată prin mecanismele de rutare ale platformei) către un nou panou de vizualizare, asigurând astfel un flux operațional continuu și predictibil.

ORARUL MEU PROGRAMĂRILE MELE LA SECRETARIAT Platformă Administrativă Facultate EN

← ÎNAPOI

Programare la Secretariat

✓ Programarea dumneavoastră a fost înregistrată cu succes. Veti fi redirectionat catre pagina Programarile Mele pentru a putea vizualiza noua programare. X

Pasul 1: Alege Data și Tipul Solicității

Data
06/30/2026

Tipul Solicității
Adeverință

Descriere
Pentru a servi la Casa de Asigurati de Sanatate

Pasul 2: Alege un Interval Orar

Dimineață

09:00	09:15	09:30
09:45	10:00	10:15
10:30	10:45	11:00
11:15	11:30	11:45
12:00	12:15	12:30
12:45		

CONFIRMĂ PROGRAMAREA

Figura 4.5 – Interceptarea răspunsurilor de succes (HTTP 2xx) și gestionarea feedback-ului asincron

4.5. Vizualizarea datelor și interfața administrativă

4.5.1. Scopul analizei vizuale in gestiunea resurselor

Obiectivul principal al acestei platforme administrative este de a reduce și de a eficientiza munca depusă de personalul universitar atunci când sunt analizate date ale sistemului. Cu toate că tabelele de date ordonate sunt extrem de eficiente pentru operațiuni cu un număr limitat de date (cum ar fi aprobarea unei anumite rezervări), acesta devin ineficiente pentru lucrul cu un număr mare de date. Pentru a le putea oferi administratorilor o prezentare generală a utilizării platformei, o interfață dedicată pentru vizualizarea de date a fost implementată, care transformă fluxul de date primit de la server în mai multe reprezentări grafice interactive.

4.5.2. Agregarea datelor in backend (\$group)

Pentru a genera aceste grafice fără a obliga clientul să efectueze calcule costisitoare ca timp și ca resurse, procesarea de date este realizată de motorul de agregare al MongoDB. Server-ul folosește operatorul `$group` pentru a calcula rezumate statistice ale mai multor colecții de date. De exemplu, pentru a vizualiza spectrul de statusuri ale programărilor, (pending, approved, completed, expired), server-ul folosește un anumit pipeline:

```
// Backend Aggregation for Dashboard Metrics
const appointmentStats = await Appointment.aggregate([
  {
    $group: {
      _id: „$status”,
      count: { $sum: 1 }
    }
  }
]);
```

Această interogare poate concentra eficient mii de documente individuale care conțin programări într-un șir JSON bine structurat care va conține doar perechile cheie-valoare necesare (ex: [{ _id: 'pending', count: 45 }, { _id: 'completed', count: 120 }]).

4.5.3. Randare pe partea de client folosind diagrame Material UI

Pe partea de client, acest payload agregat este folosit de interfața React și randat folosind librăria `@mui/x-charts` [9]. Această extensie nativă Material UI a fost aleasă pentru a asigura consistența design-ului cu restul platformei administrative, și pentru a menține un aspect asemănător, fără a utiliza alte metode externe.

Înainte de ciclul de randare, componenta utilizează interceptorul `useEffect` pentru a prelua payload-ul primit de la server și a aplica dinamic internaționalizarea globală (i18n) etichetelor setului de date. Apoi, informațiile, acum traduse, sunt mapate direct în componenta `<PieChart>`:

```
// Frontend Dashboard Component Rendering
import { PieChart } from „@mui/x-charts/PieChart”;
import { Box } from „@mui/material”;

// ... data fetching and dynamic i18n translation logic ...

<Box sx={{ height: 300, display: „flex”, justifyContent: „center”, alignItems: „center” }}>
  <PieChart
    series={[{
      data: chartData,
      innerRadius: 50, // Configures the donut-chart aesthetic
      highlightScope: { faded: „global”, highlighted: „item” },
    }]}
  />
```

```

width={400}
height={200}
/>
</Box>

```

Configurând proprietatea `highlightScope`, nivelul de vizualizare oferă o experiență reactivă pentru utilizator. Aducând cursorul/degetul peste un segment al graficului evidențiază automat porțiunea selectată din grafic și aduce toate detaliile necesare la vedere, ajutând astfel administratorii să analizeze distribuția statusurilor programărilor. Deoarece componenta este legată de administratorul de stare React, orice modificare adusă setului de date va declanșa o re-arendare instantanee a graficului, creând astfel un spațiu administrativ interactiv.

Figura 4.6 ilustrează integrarea unui modul analitic esențial pentru interpretarea rapidă a stării resurselor administrative (în acest caz, gradul de disponibilitate a sălilor). Din punct de vedere arhitectural, interfața preia un payload JSON cu date agregate și îl mapează dinamic într-o diagramă circulară utilizând o bibliotecă dedicată de vizualizare. Surprinderea proprietății `highlightScope` (care detaliază numărul exact de 14 săli disponibile) demonstrează natura interactivă a implementării. În plus, prezența butonului de comutare a vizibilității (ASCUNDE STATISTICI SĂLI) evidențiază o gestionare eficientă a stării la nivel local. Această funcționalitate permite utilizatorului să demonteze componenta grafică din DOM atunci când informațiile analitice nu sunt necesare, eliberând spațiul vizual și optimizând experiența generală de utilizare.

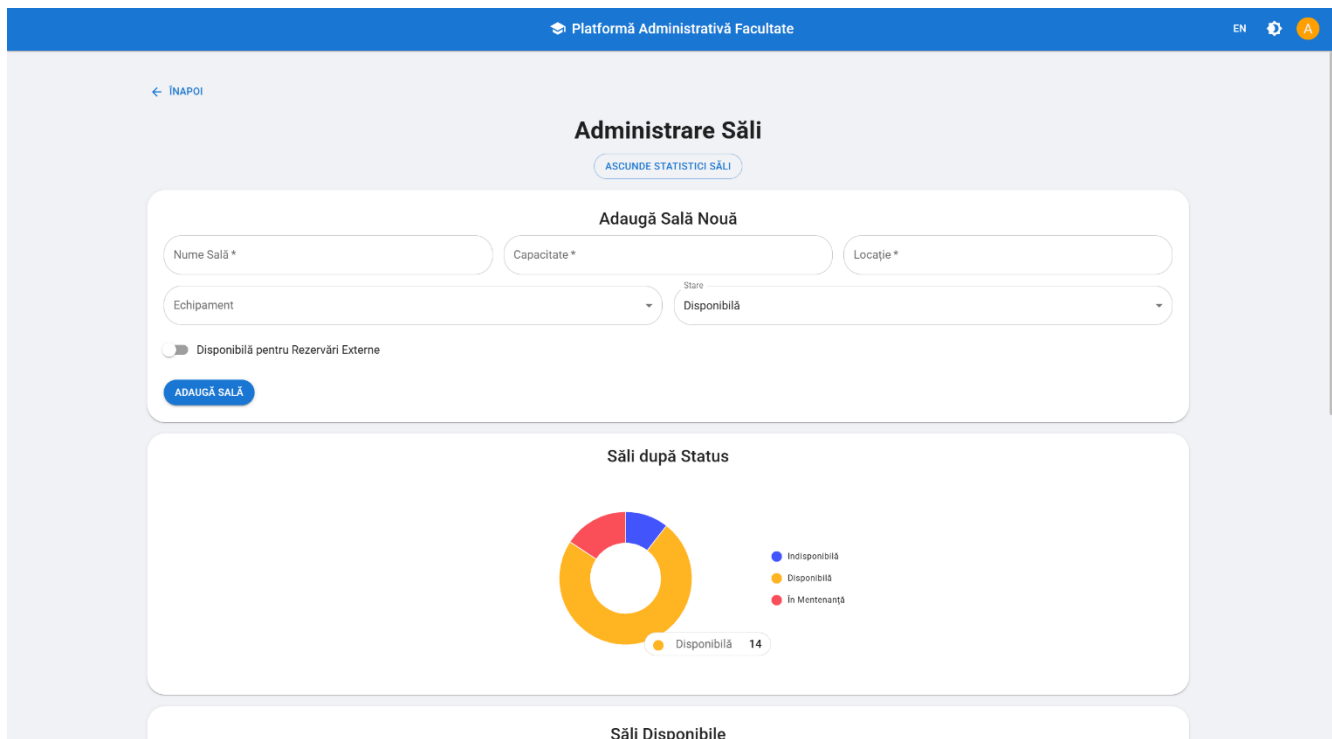


Figura 4.6 – Vizualizarea interactivă a agregărilor de stare și gestionarea stării componentelor grafice

4.6. Micro-interacțiuni și îmbunătățiri de accesibilitate

4.6.1. Protecții impuse de stare pentru acțiuni distructive

Un punct principal în proiecția interfeței de utilizator este prevenirea erorilor, în special în ceea ce privește operațiile distructive ale bazei de date (exemplu: ștergerea unui utilizator sau a unei programări). Într-un mediu web normal, o apăsare accidentală a butonului de ștergere poate cauza pierderi de date permanente. Pentru a preveni acest risc, fără a elimina conceptul de Single Page Application, o arhitectură modală de confirmare bazată pe stare a fost implementată pentru toate tabelele administrative.

Astfel, aplicația folosește interceptorul de stare `useState` pentru a suspenda acțiunea propriu-zisă, în loc să execute apelul API imediat când evenimentul `onClick` este declanșat. Sistemul prinde identificatorul unic al entității și declanșează o stare booleană pentru a putea randa o componentă `<Dialog>` a Material UI deasupra ferestrei active:

```
// React State Management for Destructive Actions
const [openDeleteModal, setOpenDeleteModal] = useState(false);
const [userToDelete, setUserToDelete] = useState(null);

const handleDeleteClick = (user) => {
  setUserToDelete(user);
  setOpenDeleteModal(true); // Suspends action and mounts the modal
};
```

Acest model arhitectural necesită o interacțiune fizică cu utilizatorul, pentru a putea confirma scopul acestuia. Acest modal este integrat în pipeline-ul `react-i18next`, astfel randând dinamic avertismentele localizate. Abia după obținerea confirmării în interiorul acestui modal clientul poate trimite cererea de ștergere, utilizând protocolul HTTP, către controlerul din server, garantând în acest mod integritatea datelor.

4.6.2. Sincronizarea ferestrei de vizualizare a utilizatorului și bucle de feedback

În timpul testării, a fost identificată o problemă vizuală de feedback în interfața administrativă. Interfața utilizatorului folosește o amplasare împărțită: formularul de manipulare a datelor (creare/editare) este ancorat în partea de sus a DOM-ului, cât timp tabelul ordonat cu date este situat în partea de jos a ferestrei de vizualizare a utilizatorului. Din acest motiv, pe ecranele mai mici (de telefon/laptop), ajungând la partea inferioară a tabelului de pe pagină, formularul din partea de sus a ferestrei este „împins” în afara ecranului. În consecință, atunci când un utilizator apasă pe butonul de editare pe un rând din tabel apropiat de partea inferioară a paginii, formularul este populat corect cu datele cerute, dar utilizatorul nu primește un feedback vizual, deoarece formularul a fost randat în afara ecranului său.

Pentru a rezolva această decuplare spațială, a fost proiectat un mecanism de sincronizare a ferestrei utilizatorului. Interfața grafică a fost îmbunătățită cu API-ul nativ DOM `Window` [14], integrat fără probleme în gestionatorul de evenimente React. În momentul în care acțiunea de a edita este

declanșată, sistemul actualizează starea componentei localizate și generează o animație care mută poziția cursorului din browser înapoi în poziția absolută de început (cât de sus posibil):

```
// Viewport Synchronization Logic
const handleEditClick = (user) => {
  // Populate the form state with the selected entity
  setFormData({
    username: user.username,
    email: user.email,
    role: user.role,
    // ... additional fields
  });
  setEditingId(user._id);

  // Force DOM synchronization to provide visual feedback
  setTimeout(() => {
    titleRef.current?.scrollIntoView({ behavior: 'smooth', block: 'start' });
  }, 100);
};
```

Aplicând parametrul `behavior: 'smooth'`, schimbarea de context este realizată cursiv, față de mutarea instantanee a ferestrei utilizatorului, lucru care ar putea crea dezorientare spațială. Micro-interacțiunile aduc un plus pentru experiența utilizatorului, făcând trecerea de la manipularea datelor neprelucrate din baza de date la un design software profesional și receptiv în fereastra utilizatorului.

4.7. Securitatea pe partea de client și arhitectura de securizare a rutelor

4.7.1. Necesitatea controlului accesului într-o aplicație full-stack

Într-o SPA decuplată, securizarea API-urilor din server folosind JSON Web Tokens este obligatorie, dar nu suficientă pentru o experiență completă a utilizatorului. În cazul în care partea de client nu are propriul nivel de securitate, un utilizator neautorizat (exemplu: un student) ar putea manipula adresa URL a browserului (exemplu: pentru a naviga către `/admins/users`) și accesa interfața administrativă a aplicației. Chiar dacă acțiunea în sine de primire a datelor va fi blocată de server, expunerea interfeței administrative reprezintă o breșă de securitate și degradează experiența utilizatorului.

Pentru a impune o securitate strictă, o arhitectură RBAC pe parte de client a fost implementată folosind `react-router-dom` [13] în componenta principală `App.jsx`.

4.7.2. Componenta `<ProtectedRoute>`

Nucleul acestui nivel de securitate la nivel de client este o componentă personalizată Higher-Order Component (HOC) numită `<ProtectedRoute>`. Această componentă se comportă ca un middleware de direcționare. Înainte ca Router-ul React să poată randa pagina cerută de utilizator,

componenta `<ProtectedRoute>` interceptează evenimentul de navigație și evaluează starea de autentificare a utilizatorului folosind API-ul Context.

Componenta este proiectată să proceseze doi parametri distincți de securitate: starea de autentificare (este utilizatorul curent autentificat?) și starea de autorizare (deține utilizatorul rolul necesar pentru a putea vizita pagina?).

```
// Implementation of Protected Route Middleware
import React from 'react';
import { Navigate, Outlet } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

const ProtectedRoute = ({ allowedRoles }) => {
  const { user, isAuthenticated, loading } = useAuth();

  if (loading) {
    // Suspend rendering until authentication is confirmed
    return <div>Loading authentication...</div>;
  }

  if (!isAuthenticated) {
    // Unauthenticated users are redirected to the login portal
    return <Navigate to="/login" replace />;
  }

  if (allowedRoles && allowedRoles.length > 0) {
    // Authenticated but unauthorized users are redirected to a safe route
    if (!user || !allowedRoles.includes(user.role)) {
      return <Navigate to="/dashboard" replace />;
    }
  }

  // If all security checks pass, render the child components
  return <Outlet />;
};
```

4.7.3. Implementare în Router-ul React

În configurația principală a aplicației (`App.jsx`), această componentă este folosită strategic pentru a izola diferite părți ale aplicației. În loc să protejeze specifice link-uri sau butoane, sistemul protejează rutele fizice URL.

De exemplu, toată suita de interfețe administrative este încapsulată într-un `<ProtectedRoute>` care cere, în mod specific, parametru de rol `admin`:

```
// Route Configuration in App.jsx
<Routes>
  {/* Public Routes */}
  <Route path="/login" element={<Login />} />
```

```

    {/* Protected Administrative Routes */}
    <Route element={<ProtectedRoute allowedRoles={[,admin']} />}>
      <Route path="/admin/dashboard" element={<AdminDashboard />} />
      <Route path="/admin/users" element={<UsersManagement />} />
    </Route>

    {/* Protected Shared Routes */}
    <Route element={<ProtectedRoute allowedRoles={[,admin', ,teacher',
,student']} />}>
      <Route path="/profile" element={<UserProfile />} />
    </Route>
  </Routes>

```

Această arhitectură declarativă de direcționare asigură că motorul React va refuza fizic să încarce componente administrative pentru profiluri neautorizate. Interceptând încercările neautorizate de navigare și efectuând o schimbare de URL fără ca utilizatorul să observe (`replace` sau `replace={true}`), aplicația păstrează o interfață de nepătruns care reflectă perfect protocoalele stricte de securitate ale server-ului Node.js.

4.8. Validare a datelor pe partea de client și prevenirea erorilor

4.8.1. Principiul apărării în profunzime

O arhitectură software robustă se bazează pe strategia apărării în profunzime. Chiar dacă serverul Node.js are rolul de a menține integritatea datelor, nu este posibilă bazarea exclusivă pe validarea pe parte de server, întrucât va fi adăugată o latență inutilă. În momentul în care un utilizator introduce un parametru invalid (exemplu: încearcă să rezerve o sală pentru zero participanți, sau la o dată din trecut), trimiterea payload-ului către server pentru a primi un răspuns HTTP 400 (cerere greșită) este o utilizare ineficientă a lățimii de bandă și scade performanța aplicației.

Pentru a optimiza eficiența sistemului, un nivel proactiv de validare pe parte de client a fost proiectat folosind componente React, pentru a intercepta date eronate înainte ca solicitarea HTTP să fie construită și, deci, trimisă către server.

4.8.2. Constrângeri proactive ale interfeței utilizatorului

Primul nivel de validare pe parte de client este implementat direct la nivel de DOM folosind standardele HTML5 și diferitele proprietăți ale componentelor Material UI. Astfel, interfața respinge intrările invalide imediat, în loc să-i permită utilizatorului să genereze o eroare, ca apoi să-l corecteze.

Ca exemplu, înăuntrul modulului de rezervare a sălilor, sistemul previne erorile temporale aplicând proprietatea `disablePast` componenteii `DatePicker`. În acest fel, utilizatorul nu va putea selecta o dată din trecut în formularele prezente în aplicație. Similar, folosind proprietățile DOM (`inputProps={{ min: 1, step: 1 }}`), se asigură faptul că numerice introduse vor fi numere strict naturale, pentru numărul de participanți la un anumit eveniment.

4.8.3. Validare logică pe bază de stări

Al doilea nivel de validare pe parte de client include verificări logice efectuate în cadrul gestionatorilor de stare ai componentei. Deoarece validările temporale necesită combinarea a obiecte diferite de date și timp, constrângerile native ale interfeței utilizatorului nu sunt suficiente.

Când secvența de trimitere a datelor este declanșată, clientul oprește execuția pentru a construi un obiect unificat `Date`, folosind parametrii de stare localizați. Apoi, compară data compusă cu ora/data utilizatorului:

```
// Frontend Temporal and Type Validation
const handleReservationSubmit = async () => {
  // Reset localized error state
  setModalError('');

  const now = new Date();
  const combinedDateTime = new Date(reservationData.date);
  combinedDateTime.setHours(
    reservationData.startTime.getHours(),
    reservationData.startTime.getMinutes(),
    0, 0
  );

  // Block past reservations bypassing the UI
  if (combinedDateTime < now) {
    setModalError({key: 'reservation_error_past_time', type: 'error'});
    return;
  }

  // Strict type and boundary checking
  const attendeesNumber = Number(reservationData.attendees);
  if (!Number.isInteger(attendeesNumber) || attendeesNumber < 1) {
    setModalError({key: 'attendees_integer_error', type: 'error'});
    return;
  }

  // Proceed to dispatch the API request
  try {
    // ... payload construction and Axios POST
```

Impunând validări stricte de tip (`Number.isInteger`) și evaluări de limite direct în browser, clientul funcționează ca un firewall extrem de eficient. Această arhitectură reduce semnificativ stresul pus pe motorul bazei de date, minimizează numărul de apeluri API, și îi furnizează utilizatorului feedback instant, alcătuit din mesaje de eroare localizate, maximizând astfel utilizabilitatea platformei administrative.

Concluzii

În concluzie, această lucrare de licență a demonstrat cu succes viabilitatea, eficiența și necesitatea implementării unei soluții software centralizate pentru administrarea ecosistemului universitar. Prin tranziția de la procese manuale, predispuse la erori umane, la o platformă digitală automatizată, sistemul rezolvă blocajele operaționale tradiționale și oferă o sursă unică de adevăr pentru toți utilizatorii implicați: studenți, profesori, administratori și reprezentanți externi.

Din perspectivă arhitecturală, utilizarea stivei tehnologice MERN s-a dovedit a fi alegerea optimă pentru dezvoltarea acestei aplicații de tip SPA. Menținerea unui ecosistem lingvistic unificat bazat pe JavaScript a permis o comunicare fluidă și rapidă între client și server. Unul dintre cele mai importante rezultate ale proiectului îl reprezintă proiectarea și implementarea algoritmului complex de detectare a suprapunerilor. Acest mecanism asigură integritatea orarului academic prin prevenirea matematică a conflictelor de alocare la nivel de sală, cadru didactic și grupă de studenți.

Mai mult, decuplarea strictă a interfeței grafice (React.js) de logica de business (Node.js/Express) și securizarea prin middleware JWT garantează un nivel de securitate de tip enterprise (RBAC). În cele din urmă, orchestrarea întregii infrastructuri prin intermediul Docker asigură portabilitatea totală a platformei, permițând implementarea acesteia pe orice mașină gazdă, eliminând astfel conflictele de dependențe.

Dincolo de implementarea tehnică propriu-zisă, impactul practic al unei astfel de soluții asupra mediului academic este unul semnificativ. Automatizarea sarcinilor repetitive și digitalizarea completă a fluxurilor de lucru reduc considerabil birocrăția și timpul acordat de către personalul administrativ gestionării datelor fragmentate. În plus, datorită arhitecturii decuplate și utilizării bazei de date NoSQL, sistemul este inerent scalabil. Aceasta înseamnă că platforma poate acomoda cu ușurință o creștere exponențială a numărului de utilizatori sau adăugarea de noi departamente și specializări în anii universitari următori, fără a necesita refactorizări la nivelul structurii de bază.

De asemenea, din perspectiva dezvoltării personale, realizarea acestui proiect a oferit oportunitatea consolidării cunoștințelor de inginerie software aplicată. Parcurgerea riguroasă a întregului ciclu de viață al dezvoltării software, de la analiza inițială a cerințelor și proiectarea arhitecturii, până la scrierea logicii backend, implementarea interfeței și orchestrarea prin containere Docker, a simulat cu succes standardele de producție din industria IT actuală. Această experiență demonstrează tranziția de la concepte teoretice la livrarea unui produs software complet, stabil și pregătit pentru utilizare în lumea reală.

Direcții viitoare de dezvoltare

Deși platforma curentă îndeplinește toate obiectivele inițiale propuse și oferă un mediu administrativ robust, arhitectura modulară a sistemului permite extinderea facilă a funcționalităților. Direcțiile viitoare de dezvoltare pentru îmbunătățirea și scalarea aplicației includ:

- **Sistem de notificări în timp real:** Integrarea protocolului WebSockets (de exemplu, prin biblioteci precum Socket.io) pentru a oferi utilizatorilor alerte instantanee. Astfel, studenții și

profesorii ar putea fi notificați în timp real atunci când o cerere de secretariat este aprobată, când apare o modificare neprevăzută în orar sau când o sală a fost rezervată cu succes.

- **Integrarea notificărilor prin email și SMS:** Implementarea unor servicii de mesagerie externe (precum Nodemailer) pentru a trimite notificări automate pe adresele de email instituționale, sporind astfel trasabilitatea operațiunilor administrative.
- **Sincronizare cu calendare externe:** Dezvoltarea unor module de import/export (CSV) și integrarea cu API-uri externe (precum Microsoft Outlook) pentru a permite utilizatorilor să își sincronizeze orarul academic direct în aplicațiile lor personale de gestionare al timpului.
- **Generare automată a orarului (inteligență artificială):** Extinderea algoritmului curent de detectare a suprapunerilor într-un motor de optimizare bazat pe algoritmi euristici (de exemplu, algoritmi genetici). Sistemul ar putea nu doar să valideze datele introduse manual, ci să genereze automat variante optime de orar, ținând cont de preferințele și constrângerile tuturor profesorilor și de capacitatea maximă a infrastructurii fizice.
- **Aplicație mobilă nativă:** Utilizând codul și componentele deja scrise în React.js, interfața frontend ar putea fi migrată către React Native, oferind o aplicație mobilă dedicată (iOS și Android). Aceasta ar asigura o experiență de utilizare și mai accesibilă pentru studenți, valorificând funcțiile hardware native ale telefoanelor.

Bibliografie

- [1] Meta Open Source, Documentație React: librăria utilizată pentru aplicații web și creare de interfațe native pentru utilizatori,
<https://react.dev/>
Accesat la 25.03.2026
- [2] MongoDB Inc, Manual MongoDB,
<https://www.mongodb.com/docs/manual/>
Accesat la 27.03.2026
- [3] OpenJS Foundation, Documentație Node.js și documentație Motor V8
<https://nodejs.org/learn/getting-started/the-v8-javascript-engine>
Accesat la 02.04.2026
- [4] Introducere JSON Web Tokens,
<https://www.jwt.io/introduction#what-is-json-web-token-structure>
Accesat la 30.03.2026
- [5] MongoDB Inc, Condiții și interogări de filtrare,
<https://www.jwt.io/introduction#what-is-json-web-token-structure>
Accesat la 12.05.2026
- [6] node-cron: Planificator de activități în fundal scris în JavaScript pentru NodeJS,
<https://www.npmjs.com/package/node-cron>
Accesat la 13.05.2026
- [7] MongoDB Inc, Etapele scrierii unui pipeline de agregare - \$facet,
<https://www.mongodb.com/docs/manual/reference/operator/aggregation/facet/>
Accesat la 20.04.2026
- [8] i18next, Documentație react-i18next,
<https://react.i18next.com/>
Accesat la 02.06.2026
- [9] MUI, Documentație MUI, grafice X Charts
<https://mui.com/x/react-charts/>
Accesat la 08.06.2026
- [10] David Thomas, Andrew Hunt, *The Pragmatic Programmer: Your Journey To Mastery, 20th Anniversary Edition*, Addison-Wesley Professional, 2019
- [11] Faker, „Generate massive amounts of fake (but realistic) data for testing and development.”,
documentație Faker.js,
<https://fakerjs.dev/guide/usage.html>
Accesat la 30.05.2026
- [12] Adam Wiggins, The Twelve-Factor App, disponibil la:
<https://www.12factor.net/>
Accesat la 05.06.2026

- [13] React Router, documentație, disponibil la:
<https://reactrouter.com/home/>
Accesat la 04.06.2026
- [14] Element: scrollIntoView() method, Documentație metodă disponibilă la:
<https://developer.mozilla.org/en-US/docs/Web/API/Element/scrollIntoView>
Accesat la 06.06.2026
- [15] Docker, Documentație Docker disponibilă la:
<https://docs.docker.com/>
Accesat la 08.06.2026
- [16] Node.js, Bucla de evenimente, documentație disponibilă la:
<https://nodejs.org/learn/asynchronous-work/event-loop-timers-and-nexttick#event-loop-explained>
Accesat la 10.06.2026
- [17] JSON Web Token (JWT), ce este un JWT, și cum se folosește; documentație disponibilă la:
<https://www.jwt.io/introduction#what-is-json-web-token>
Accesat la 09.06.2026
- [18] BSON, ce este un BSON, și la ce se folosește; documentație disponibilă la:
<https://bsonspec.org/>
Accesat la 10.06.2026
- [19] Ghid de accesibilitate al conținutului web (WCAG); documentație disponibilă la:
<https://www.w3.org/TR/WCAG21/>
Accesat la 10.06.2026
- [19] Accesibilitate Material UI; documentație disponibilă la:
<https://mui.com/x/react-tree-view/accessibility/>
Accesat la 10.06.2026
- [20] Axios, ce este Axios, și cum este folosit; documentație disponibilă la:
<https://axios.rest/pages/getting-started/features.html>
Accesat la 11.06.2026
- [21] CSS-in-JS, ce este, și cum este folosit; documentație disponibilă la:
https://www.w3schools.com/react/react_css_js.asp
Accesat la 11.06.2026